2026

# Benchmarking 12 Deep Learning Models for Binary Classification: Accuracy, Complexity, and Speed

A Comprehensive Evaluation Across 182 Tabular Datasets

Ed Kaempf
April 2026

# Benchmarking 12 Deep Learning Models for Binary Classification: Accuracy, Complexity, and Speed

## Executive Summary

This study benchmarks 12 deep learning models on 182 tabular datasets for binary classification (predicting which of two possible outcomes applies to a given case) to answer four questions: (1) Which models perform best overall? (2) What makes datasets difficult to classify? (3) Which models handle specific complexity types most effectively? (4) How do accuracy and speed trade off across models?

### *Model Performance*

Eleven of the 12 models form a statistically equivalent top tier, with mean accuracy ranging from 77.1% to 83.8%. While no single adjacent pair differs significantly, the full spread of 6.7 percentage points across the group is not trivial in practice. Practitioners should consider both the statistical grouping and the absolute accuracy range when selecting a model, weighing factors such as throughput, training cost, and hardware constraints alongside performance. TabTransformer is the sole exception, standing significantly below the field on every metric evaluated including accuracy, win rate, and probability calibration.

When accuracy and prediction speed are combined into a single deployment score, Category Embedding and TabMLP are the strongest all-around choices, delivering both high accuracy and fast inference. FT-Transformer achieves the highest number of dataset-level wins, but is notably slower, presenting a meaningful tradeoff for latency-sensitive applications. NODE and DANet score poorly on both dimensions and are difficult to recommend for production deployment.

A simple model selection guide is shown below:

| Situation | Recommended Model |
|---|---|
| 1. Best all-around deployment | Category Embedding or TabMLP |
| 2. Accuracy is the only priority, speed is not a constraint | FT-Transformer |
| 3. Avoid in production | NODE, DANet, TabTransformer |

Prediction speed varied by more than 30-fold across the 12 models. Differences of this size are large enough to make some models impractical in real-time or high-volume applications regardless of how accurate they are.

### *ML vs. DL*

Across 158 shared datasets, traditional ML models occupy the top seven accuracy positions, with XGBoost, Random Forest, and LGBM Classifier holding the top three spots, with a narrow 0.2% spread. When ranked by win rate, XGBoost leads at 15.2%. The best DL models are competitive

with mid-tier ML, as FT-Transformer matches Random Forest at rank 6, but have not closed the gap with top ML methods. Notably, model selection within each approach matters as much as the choice between ML and DL.

### *What Makes Datasets Hard to Classify*

Neighborhood complexity measures (N1, N3, N4) are the strongest predictors of classification difficulty, with correlations against accuracy of -0.87, -0.83, and -0.80. Linearity measures (L2, L3) follow closely. Class imbalance, contrary to common assumption, shows a positive correlation with accuracy in this collection, a result explained by the fact that highly imbalanced datasets here tended to have simpler geometric structure. Dataset geometry is a more reliable predictor of difficulty than class distribution or dataset size.

### *Novel Finding*

Runtime profiling of the 22 Lorena et al. complexity measures shows that the clsCoef measure alone accounts for 81% of total time needed to calculate these measures on each dataset. This contradicts prior claims that neighborhood and tree-based measures as the primary bottlenecks. This finding has practical implications for any researcher computing these measures at scale.

# 1. Introduction

This section is organized as follows:

1.1 Overview and Motivation

1.2 Dataset Complexity and Model Performance

1.3 Tabular Datasets

1.4 Report Organization.

## 1.1. Overview and Motivation

The primary goal of a machine learning model is to make accurate predictions on new, unseen data. Machine learning practitioners consistently face a fundamental challenge: selecting which algorithm will perform best on a specific dataset. Traditional benchmarking studies evaluate models across multiple datasets but rarely examine how dataset characteristics affect algorithm effectiveness. As a result, practitioners often rely on trial-and-error experimentation with limited insight into why certain algorithms succeed or fail.

This study addresses this gap by systematically linking algorithm performance to measurable dataset complexity. By characterizing 182 binary classification datasets across 22 complexity dimensions and evaluating 12 diverse deep learning models, this work shows which complexity factors drive classification difficulty, and which models manage specific complexity types most effectively. These findings enable evidence-based algorithm selection tailored to dataset characteristics rather than generic "best practices" that may not apply to specific problem contexts.

### 1.2. Dataset Complexity and Model Performance

Dataset complexity measures describe how hard a dataset is to classify. The measures span geometric structure, feature relationships, class separability,[1] and data overlap.

Ho and Basu in 2002 [1], introduced 12 complexity measures to characterize the difficulty. Ho and Basu defined the 12 measures, grouping them into overlap measures (F1–F3), separability measures (L1–L2, N1–N3), and geometry/overlap measures (L3, N4, T1, T2). Together, these measures capture how features, class boundaries, and structural properties affect classification difficulty.

In their 2018 study [2], Lorena et al. added measures for feature correlation, linearity, neighborhood, network, dimensionality, and class balance. Lorena et al. developed 22 measures of dataset complexity, organized in six categories.

This project applies all 22 measures to characterize dataset complexity. These measures capture what makes a dataset hard to classify. [3]

### 1.3. Tabular Datasets

Tabular datasets, organized as rows of samples and columns of features, underpin mission-critical applications across healthcare, finance, manufacturing, retail, and many other domains. Use cases span fraud detection, credit risk analysis, customer churn prediction, medical diagnosis, and quality control, among others.

Despite their ubiquity, tabular datasets pose distinct challenges that affect model performance, including missing values, mislabeled records, categorical features, class imbalances, high dimensionality, redundant features, outliers, and noise. These characteristics vary widely across datasets, making fair model comparison difficult without consistent, systematic testing. Unlike image or text domains where standardized benchmarks are well established, tabular data require dedicated benchmarking to evaluate model performance under realistic conditions.

### 1.4. Report Organization

The remainder of this report is organized as follows:

- **Section 2, Methodology** – machine learning models evaluated, datasets used for benchmarking, dataset complexity measures, evaluation procedures, and benchmarking workflow
- **Section 3, Performance Metric Validation** – correlations between accuracy, F1, and AUC, with justification for using accuracy as the primary metric
- **Section 4, Model Performance and ML vs. DL Comparison** – DL model performance results across all 182 datasets, and ML and DL performance across 158 shared datasets

[1] In binary classification, a 'class' refers to one of the two possible outcomes the model is trying to predict. For example, in a fraud detection dataset, the two classes would be 'fraudulent' and 'legitimate'.

- **Section 5, Dataset Complexity Analysis** – dataset complexity characteristics and their impact on performance
- **Section 6, Model Performance by Dataset Characteristic** – model performance examined by complexity measure.
- **Section 7, Model Performance vs. Throughput** – throughput results and accuracy–throughput tradeoffs
- **Section 8, Conclusion** – summary of results and key takeaways
- **Section 9, Limitations and Future Work** – constraints of this study and directions for extension.

**Appendices** provide supporting details:

- **Appendix A, Dataset Complexity Measures** – descriptions of each measure, organized by the six categories of Lorena et al. measures [2]
- **Appendix B, Dataset Characteristics and Sources** – full list of datasets with name, records, features, source, and URL
- **Appendix C, Computational Infrastructure** – Python environment, hardware specifications, and code availability statement
- **Appendix D, Pearson Correlations Between Complexity Measures and Accuracy by Model Family** – Pearson correlations between all 22 complexity measures and accuracy computed separately for each of the four model families.

Finally, the **References** section lists all cited studies.

## 2. Methodology

This section is organized as follows:

2.1 Deep Learning Models Evaluated

2.2 Datasets Used for Benchmarking

2.3 Dataset Complexity Measures

2.4 Evaluation Procedures.

### 2.1. Deep Learning Models Evaluated

This study evaluated 12 deep learning models selected to represent a broad cross-section of classification algorithms. These models span diverse learning strategies, from simple linear classifiers to complex ensemble methods, showing the range of techniques available for structured tabular data.

### *Strategic Model Selection*

To ensure findings are directly applicable to real-world enterprise environments, this study evaluates 12 deep learning models selected for their stability, wide adoption, and production readiness. We chose established architectures supported by industry-standard frameworks, including *PyTorch Tabular* and *PyTorch WideDeep*. The selection emphasizes established, production-ready algorithms rather than experimental or domain-specific approaches, ensuring findings are directly applicable to real-world classification tasks..

### *The Architectural Taxonomy*

The 12 models are organized into four "families" based on their core design logic. This taxonomy allows for a systematic comparison of how different mathematical structures manage varying dataset complexities:

- **Transformer-based:** Architectures that use "self-attention" to find complex patterns and relationships across all input features simultaneously
- **Tree-Neural Hybrids:** Models that blend the logical, rule-based strengths of traditional Decision Trees with the high-performance learning of Neural Networks
- **Feature Interaction / Attention:** Specialized models designed to focus specifically on how pairs of data points interact, offering a targeted alternative to full sequence modeling
- **MLP-based (Multi-Layer Perceptron):** The robust foundation of deep learning, using high-performance feed-forward layers and optimized embedding support for tabular data.

### *Readability & Convention*

For clarity in reporting, model names have been streamlined. Technical prefixes (such as "WideDeep_") are omitted where context allows, and industry-standard abbreviations (e.g., SAINT, GATE, NODE) are used throughout the exhibits to help with quick comparison. The 12 models, categorized by family, are the following:.

**A. Transformer-based**

1. FT-Transformer
2. TabTransformer
3. SAINT
4. Self-Attention MLP
5. TabFastFormer

**B. Tree-Neural Hybrid**

1. GANDALF
2. GATE
3. NODE

**C. Feature Interaction/Attention**

1. AutoInt
2. DANet

**D. MLP-based**

1. Category Embedding
2. TabMLP.

The four families reflect different approaches to learning from tabular data:

**A. Transformer-based**: models that compare all input features to one another using attention, a mechanism originally developed for language models. This allows them to capture complex relationships across the entire feature set.

**B. Tree-Neural Hybrid**: models that combine the rule-based logic of decision trees with the learning power of neural networks.

**C. Feature Interaction/Attention**: models designed to identify important interactions between pairs or small groups of features, rather than treating all features equally. They focus on learning which feature combinations matter most.

**D. MLP-based**: models built on the classic feed-forward neural network structure, which passes data through successive layers of calculations to learn patterns.

## 2.2. Datasets Used for Benchmarking

### *Dataset Characteristics*

This project used 182 publicly available datasets selected based on three criteria:

- A target field existed and had no more than three values (later binarized to two)
- No more than 5% of the records had missing values
- Did not involve games like poker or chess (where the target is a deterministic function of the data).

Of the 182 datasets, 169 had no missing values, and 13 had less than 5% missing values. All are available from repositories of datasets not intended to represent the diverse, unclean, and complex data found in real-world datasets. Many are preprocessed before release (e.g., missing values removed, outliers filtered). As noted by [4], datasets may also have well-engineered features that simplify the modelling process and enhance prediction performance.

A simple measure of class imbalance is the ratio of records in the majority class to those in the minority class. A perfectly balanced dataset has a ratio of 1.0, while larger values indicate increasing imbalance. Across the 182 datasets, the median imbalance was 1.68, with values ranging from 1.0 to 773.4 (online_payments_fraud_detection dataset).

Fifty-three datasets included categorical features; the remaining 129 did not. All categorical features were one-hot encoded. Encoding produced between 2 and 21,848 features per dataset, with an average of 378 encoded features.

Due to time limits, the 182 datasets represent a small set of all datasets meeting the selection criteria. The full list of datasets is provided in **Appendix B**. **Figure 2.1**, on the following page, summarizes the sources from where the datasets were obtained. [2]

[2] The four sources of the datasets include: OpenML, UCI Machine Learning Repository, Kaggle, and Scikit_Learn .

**Figure 2.1 Dataset Sources**

| Source | Count |
|---|---|
| OpenML | 131 |
| UCI | 30 |
| Kaggle | 15 |
| Other | 6 |
| **Total** | **182** |

**Figure 2.2** profiles of the 182 datasets. The model benchmarks capture performance across both small and large, simple and complex datasets.

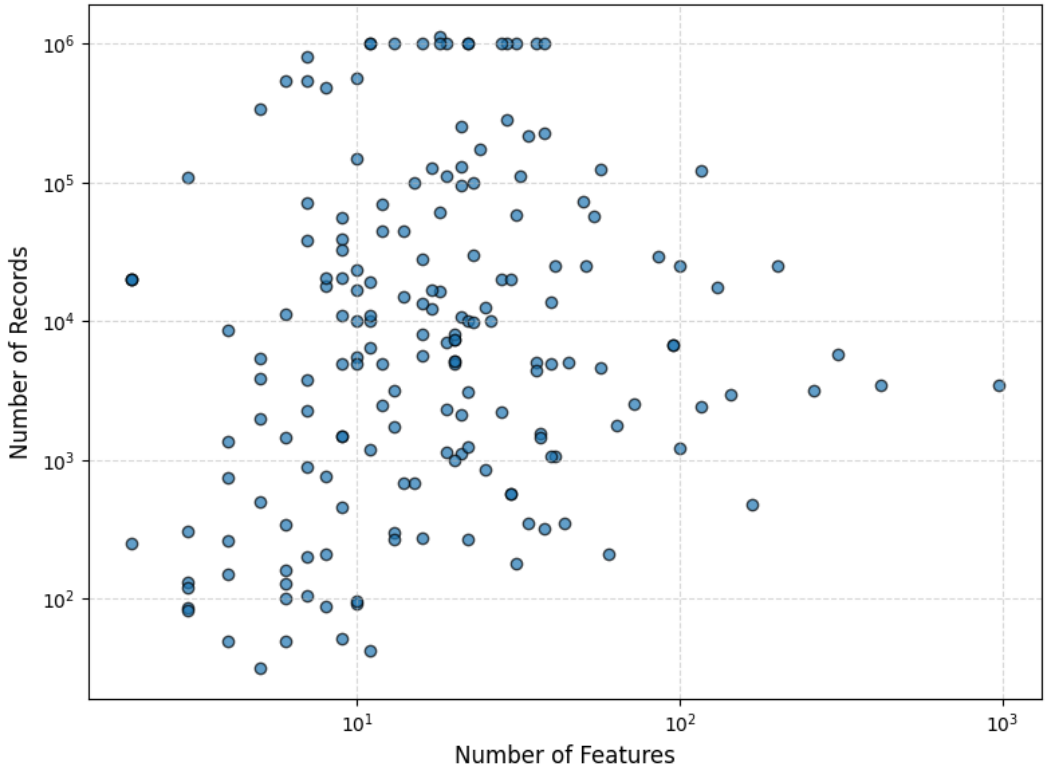
Three of the four smallest datasets, with two features each, were the three datasets from scikit-learn's suite of toy datasets[3]. The malware classification EMBER2024 dataset available on GitHub has 2,568 features and 3.2 million labeled samples. Because of a compute limit, 50,000 samples were randomly selected for benchmarking. **Figure 2.3**, on the following page, illustrates dataset size and dimensionality, shown on a logarithmic scale for both record counts and feature counts.

**Figure 2.2 Dataset Characteristics: Size and Feature Counts**

| Attribute | Count |
|---|---|
| Record Count Median | 6,658 |
| Record Count Range | 32 – 1,138,289 |
| Feature Count Median | 17 |
| Feature Count Range | 2 – 2,569 |
| ***Numeric Features:*** | |
| Count Mean | 48 |
| Count Median | 14 |
| Count Range | 1 – 2,568 |
| ***Categorical Features:*** | |
| Datasets with at Least One Categorical | 53 |
| Count Mean: | |
| All 182 Datasets | 1 |
| The 53 with At Least One Categorical | 5 |
| Count Median: | |
| All 182 Datasets | 0 |
| The 53 with At Least One Categorical | 3 |
| Encoded Count Median | 26 |
| Encoded Count Range | 2 – 21,848 |

[3] The three scikit-learn toy datasets are: make_blobs, make_circles, and make_moons.

**Figure 2.3 Dataset Characteristics: Number of Records and Dimensionality ($\log_{10}$)**




### *Dataset Preprocessing Procedures*

Each dataset underwent a consistent preprocessing pipeline prior to model training. Column names were standardized by replacing special characters with underscores to prevent model errors. Embedded quotes were stripped from string values in categorical columns. Features were classified as either numeric or categorical based on their data type. Missing values in numeric columns were imputed using the median strategy, while categorical columns were left for the model frameworks to handle natively. Numeric features were converted to a standard numeric format (float32) required by PyTorch. The target variable was label-encoded to binary values (0 or 1) using scikit-learn's LabelEncoder. All preprocessing parameters were fit on the training split only and applied separately to the test split to prevent data leakage.

Categorical features were identified and passed to the *pytorch_tabular* framework, which handles them internally through learned embedding layers during model training rather than applying a fixed encoding scheme such as one-hot or ordinal encoding.

The four *pytorch-widedeep* models used the same preprocessing pipeline as the *pytorch_tabular* models, with two additions: numeric features were standardized to zero mean and unit variance, and categorical features were explicitly prepared for embedding using the *pytorch-widedeep* TabPreprocessor prior to training.

## 2.3. Dataset Complexity Measures

Complexity measures quantify dataset traits that affect classification difficulty. They capture aspects of geometric structure, feature relationships, class separability, and data overlap. This project employs the Lorena et al. (2019) framework, [2] which defines 22 measures organized into six categories: feature-based, linearity, neighborhood, network, dimensionality, and class imbalance. By correlating these measures with model performance, we show which dimensions most strongly predict classification difficulty and which models show robustness.

Ho and Basu (2002), [1] introduced 12 measures focused on the geometric characteristics of class distributions. They emphasized that the way classes are separated or interleaved is critical for classification accuracy. Their measures fell into three groups:

- Overlap of feature values: Fisher's discriminant ratio (F1), Volume of overlap region (F2), Feature efficiency (F3).
- Separability: Linear separability (L1, L2), Mixture identifiability (N1, N2, N3).
- Geometry, topology, and density: Nonlinearity (L3, N4), Proportion of hyperspheres covering data (T1), Average number of samples per feature (T2).

Lorena et al. [2] expanded the framework to 22 measures, to include feature correlation, linearity, neighborhood, network, dimensionality, and class balance. These 22 measures, considered state-of-the-art for detecting class overlap and classification difficulty, are summarized in **Exhibit 2.1**, on the following page. Full names, acronyms, and descriptions are provided in **Appendix A**.

**Figure 2.5** illustrates the six complexity categories with stylized examples, adapted but then modified from [5] . Low-complexity datasets (top row) show clear class separation and simple decision boundaries, while high-complexity datasets (bottom row) show overlapping classes, non-linear boundaries, and intricate topological structures. These visual differences translate into values calculated for the 22 Lorena et al. measures.

**Figure 2.5 Examples of Low and High Dataset Complexity by Category**

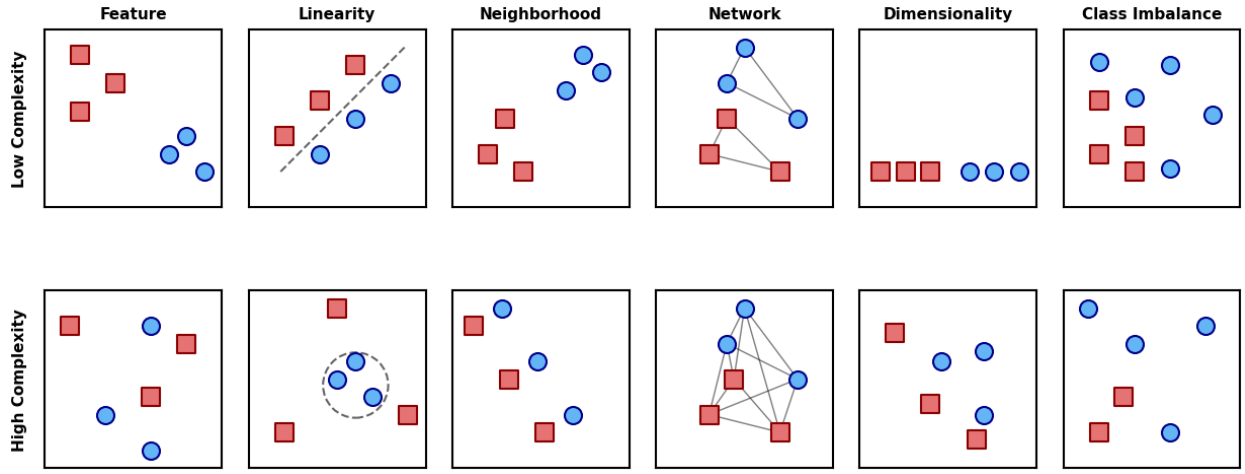


**Exhibit 2.1 Dataset Complexity Measures**

| Complexity Measures | |
|---|---|
| **Feature-based**<br>• **F1**: Maximum Fisher's discriminant ratio – overlap between the values of the features in different classes<br>• **F1v**: Directional-vector Maximum Fisher's Discriminant Ratio – version of F1 that searches for a separating vector<br>• **F2**: Volume of overlapping region – overlap of the distributions of the features values within the classes<br>• **F3**: Maximum individual feature efficiency – maximum value of most discriminative feature<br>• **F4**: Collective feature efficiency – combined discriminative power of features<br>**Linearity**<br>• **L1**: Sum of the error distance by linear programming – ability to separate classes with a linear boundary.<br>• **L2**: Error rate of linear classifier – misclassification rate under linear separation.<br>• **L3**: Nonlinearity of a linear classifier – error rate on new data points<br>**Neighborhood**<br>• **N1**: Fraction of borderline points – proportion of samples near class boundaries.<br>• **N2**: Ratio of intra/extra class nearest neighbor distance – compares the distances inside a class with those between classes<br>(continued above) | **Neighborhood (continued)**<br>• **N3**: Error rate of nearest neighbor classifier – ratio of misclassified points using leave one out 1-NN<br>• **N4**: Nonlinearity of nearest neighbor classifier – ratio of the misclassified interpolated points<br>• **T1**: Fraction of hyperspheres covering data – how intermixed the different classes are<br>• **LSC**: Local set average cardinality – average of the set of points whose distance is smaller than the distance of the other class<br>**Network**<br>• **Density**: Average density of the network – density relative to dimensionality<br>• **ClsCoef**: Clustering coefficient – the tendency to create cliques<br>• **Hubs**: Network density – graph connectedness of the vertices<br>**Dimensionality**<br>• **T2**: Average number of features per points – ratio of dimensionality to number of samples<br>• **T3**: Average number of PCA dimensions per points – ratio of PCA components to number of samples<br>• **T4**: Ratio of the PCA dimension to the original dimension – proportion of relevant dataset dimensions<br>**Class imbalance**<br>• **C1**: Entropy of classes proportion– normalized entropy of the class size distribution<br>• **C2**: Imbalance ratio – and index of class imbalance |

## 2.4. Evaluation Procedures

Model training was performed in Google Colab using a high-performance NVIDIA A100 GPU (a specialized processor that dramatically accelerates deep learning computations), while all analysis and reporting were completed on a Windows PC using standard Python data science tools. Each model followed a consistent train-validation-test process with safeguards such as early stopping to ensure reliable and efficient training. Throughput was measured under identical conditions across all models, providing a fair basis for comparing their computational performance.

Training was capped at 40 epochs (one epoch is a single complete pass through the training data) for all models. Early stopping was applied in both frameworks, monitoring validation loss with a patience of 5 epochs and a minimum improvement threshold of 0.001. Each model was trained on one portion of the dataset, and its accuracy was measured by predicting the target variable on a separate, unseen portion of data that was reserved solely for testing. In total, the strategy produced 2,172 performance observations (12 models × 182 datasets, minus 12 model-dataset pairs that could not complete).

Three performance metrics were recorded for each model-dataset pair:

- **Accuracy** (proportion of correct predictions)
- **F1-weighted score** (harmonic mean of precision and recall, weighted by class support)
- **AUC** (area under the ROC curve).

Preliminary analysis revealed very high correlation between accuracy and F1-weighted scores (Pearson's $r = 0.977$, $R^2 = 0.954$), indicating these two performance metrics were redundant for this dataset collection. There is a much weaker correlation between accuracy and AUC (Pearson's $r = 0.692$, $R^2 = 0.479$). Accuracy was selected as the primary performance metric due to its interpretability and the demonstrated equivalence with F1 in model rankings (detailed in Section 3).

Statistical significance testing (Friedman and Nemenyi) was used to determine whether performance differences between models were meaningful. Dataset complexity measures were used separately to explain what makes certain datasets harder to classify.

Computational efficiency was assessed using two metrics: prediction throughput (predictions per second) and peak GPU memory usage during training and inference. All measurements were taken on consistent hardware to ensure comparability, and preprocessing time was excluded so that results reflect only model execution. Throughput was measured on the test split for each dataset under identical conditions, providing a fair and representative view of each model's deployment-oriented performance.

All numerical features were standardized to zero mean and unit variance prior to model training, ensuring fair comparison across algorithms. Hyperparameters[4] were not optimized, as each model was evaluated with default settings. Because all models used default settings, the

[4] A setting that controls how a model learns, such as learning rate or number of layers. Hyperparameters are chosen before training and can strongly affect performance.

results show comparative performance under identical constraints, not the best possible performance each model could achieve with tuning

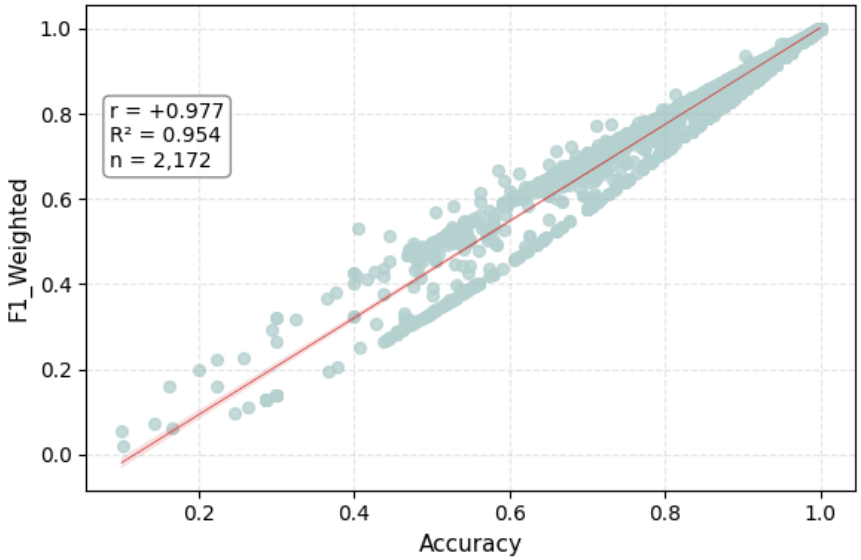
## 3. Performance Metric Validation

Each model–dataset pair was evaluated using three metrics: accuracy, F1-weighted score, and area under the ROC curve (AUC). To assess redundancy and select a primary metric, we examined correlations across all 2,172 model–dataset combinations.

Accuracy and F1-weighted scores were highly correlated (Pearson's $r = 0.977$, $R^2 = 0.954$), indicating that both metrics ranked models similarly. **Figure 3.1** illustrates this relationship, with each point being a model–dataset pair and a trend line illustrating the near-linear association.

Given the strong redundancy between accuracy and F1-weighted, accuracy was chosen as the primary metric. It is easy to interpret, aligns with F1-weighted rankings, and is robust to the range of class imbalance levels present in the 182 benchmark datasets, making it suitable for summarizing model performance across diverse datasets.

**Figure 3.1 Accuracy vs. F1-Weighted Score Across All Model–Dataset Pairs**




AUC was kept for reference but not used as the primary metric due to interpretability and availability constraints. Although AUC reflects ranking quality independent of threshold, it is less intuitive for binary tasks with primarily balanced classes.

## 4. Model Performance

This section is organized as follows:

4.1 Deep Learning Model Performance
4.2 Machine Learning and Deep Learning Performance Comparison
4.3 Statistical Significance Testing
4.4 Prediction Calibration: Brier Score Analysis
4.5 Memory Efficiency and Stability
4.6 Evaluating TabTransformer's Sensitivity to Hyperparameter Tuning.

## 4.1. Deep Learning Model Performance

**Figure 4.1** summarizes model performance across 179-182 datasets, ordered by mean accuracy.[5]

**Figure 4.1 Deep Learning Model Performance**

| DL Model | Accuracy | Median | Std. Dev. (pp) | Min. | Max. |
|---|---|---|---|---|---|
| TabFastFormer | **83.8%** | 85.6% | **12.5** | **28.6 %** | 100% |
| TabMLP | 83.7 | 86.0 | 14.1 | 14.3 | 100 |
| Category Embedding | 83.6 | **86.5** | 13.9 | 30.0 | 100 |
| AutoInt | 83.4 | 85.8 | 13.2 | 28.6 | 100 |
| FT-Transformer | 83.2 | 86.2 | 14.9 | 22.2 | 100 |
| Self-Attention MLP | 82.3 | 85.0 | 14.3 | **28.6** | 100 |
| GANDALF | 81.9 | 85.5 | 14.8 | 20.0 | 100 |
| GATE | 79.6 | 84.0 | 16.6 | **28.6** | 100 |
| SAINT | 78.2 | 82.4 | 17.2 | 10.2 | 100 |
| NODE | 78.0 | 79.6 | 15.0 | **28.6** | 100 |
| DANet | 77.1 | 83.1 | 19.7 | 16.5 | 100 |
| TabTransformer | 72.7 | 73.8 | 18.1 | 10.0 | 100 |

TabFastFormer's combination of the highest mean accuracy and the lowest standard deviation makes it the most consistently dependable model in this benchmark. Its consistently narrow performance spread across 182 datasets suggests the best level of robustness and generalization among the other models.

Six key patterns appear from these results.:

- **Comparable performance across three of four model families**: Among three of the architectures (Transformer-based, MLP-based, and Feature Interaction/Attention), none dominated. Mean accuracies across the top five models are separated by just 0.5 percentage points. The results suggest that architectural complexity alone does not determine performance on tabular binary classification.

[5] The F1 macro weighted score for each model averaged 2.4 percentage points lower than its accuracy score. The differences ranged from 0.8 percentage points lower (TabTransformer) to 4.7 percentage points lower (SAINT)

- **Tree-neural hybrids underperform expectations**: GANDALF, GATE, and NODE, are consistently below top tier, with no clear advantage over MLPs. This suggests hybridizing trees with DL does not reliably translate to better performance.
- **Feature-interaction models show mixed results**: AutoInt performs strongly (near top tier), but DANet is unstable (second lowest accuracy, worst std dev). Explicit interaction modeling can help but is not robust.
- **Comparable stability**: Stability varies moderately across all 12 models, with Std. Dev. ranging from 12.5pp (TabFastFormer) to 19.7pp (DANet). Some models are reliably consistent while others produce slightly more variable results depending on dataset characteristics. This suggests that dataset difficulty affects most algorithms similarly, rather than creating model-specific challenges.
- **Simple v. Complex**: The two MLP-based models, TabMLP and CategoryEmbedding, rank 2nd and 3rd overall, outperforming more architecturally complex Transformer and Tree-neural hybrid models, reinforcing the well-documented finding that simpler architectures remain competitive on tabular data
- **Lower baseline performance of TabTransformer**: By restricting attention to categorical features, TabTransformer excludes numeric features from its interaction modeling, which can reduce accuracy and stability. Architectures that include all features in attention, such as FT-Transformer, tend to perform better and more consistently.

**Figure 4.2**, on the following page, summarizes model ranking frequencies across the 182 datasets, reporting the number of times each model ranked first, the frequency of Top-3 finishes, and the overall win rate. Several patterns appear:

- FT-Transformer stands apart from the field, with a win rate nearly 6 percentage points higher than the next best model, suggesting it is the most reliably competitive architecture across diverse tabular datasets. It is a simple transformer model combined with a module for embedding categorical and numerical features, created in [6].
- MLP-based models outperform several more complex architectures on ranking frequency, challenging the assumption that architectural sophistication translates to better performance on tabular data.
- TabTransformer's near-absence from Top 3 finishes is notable given that other Transformer-based models dominate the top of the rankings, suggesting the architecture itself is not the determining factor and that implementation choices within the family matter considerably.
- No single architecture family monopolizes the top positions, with Transformer-based, MLP-based, and Feature-Interaction models all represented in the top 5 by win rate.

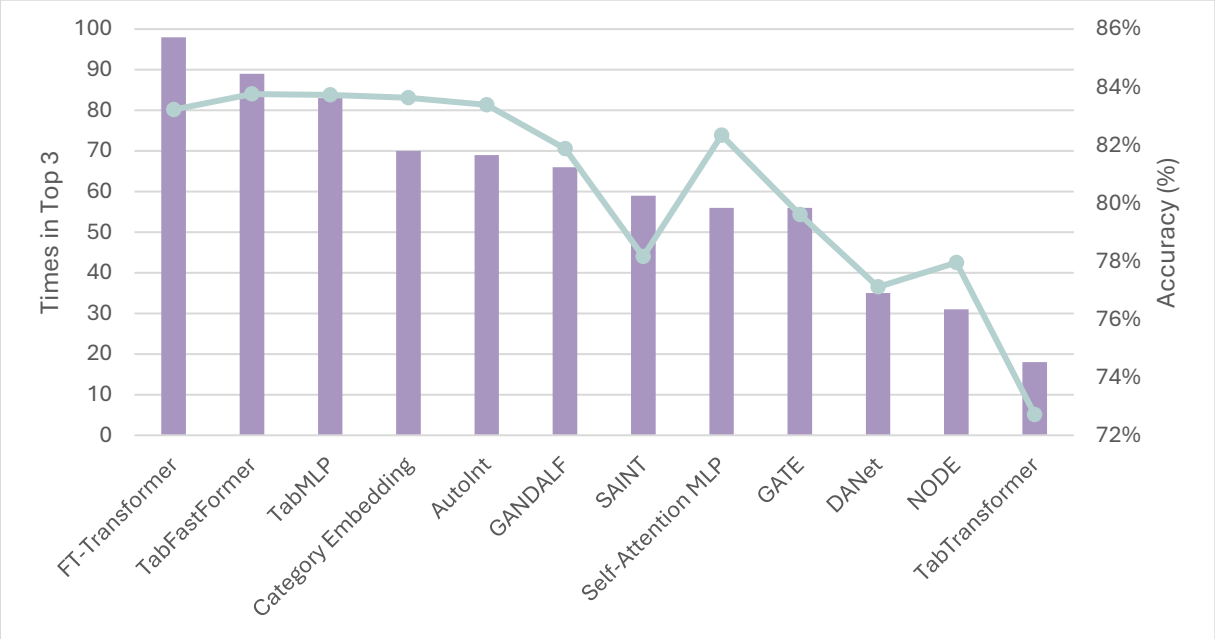
**Figure 4.3**, following this page, shows that the FT-Transformer and TabFastformer models consistently lead performance across the 182 datasets. The two rank in the Top 3 far more often than other architectures and maintain the highest overall accuracy. In contrast, models such as TabTransformer, NODE, and DANet appear less often among the top performers and achieve lower mean accuracy.

**Figure 4.2 Deep Learning Model Ranking Frequencies (in order of Times in the Top 3)**

| DL Model | Times #1 | Times Top 3 | Win Rate [a] |
|---|---|---|---|
| FT-Transformer | 49 | 98 | 26.9 |
| TabFastFormer | 38 | 89 | 20.9 |
| TabMLP | 32 | 83 | 17.6 |
| Category Embedding | 32 | 70 | 17.6 |
| AutoInt | 27 | 69 | 14.8 |
| GANDALF | 25 | 66 | 13.7 |
| SAINT | 25 | 59 | 13.7 |
| Self-Attention MLP | 16 | 56 | 8.8 |
| GATE | 24 | 56 | 13.2 |
| DANet | 19 | 35 | 10.4 |
| NODE | 13 | 31 | 7.1 |
| TabTransformer | 9 | 18 | 4.9 |

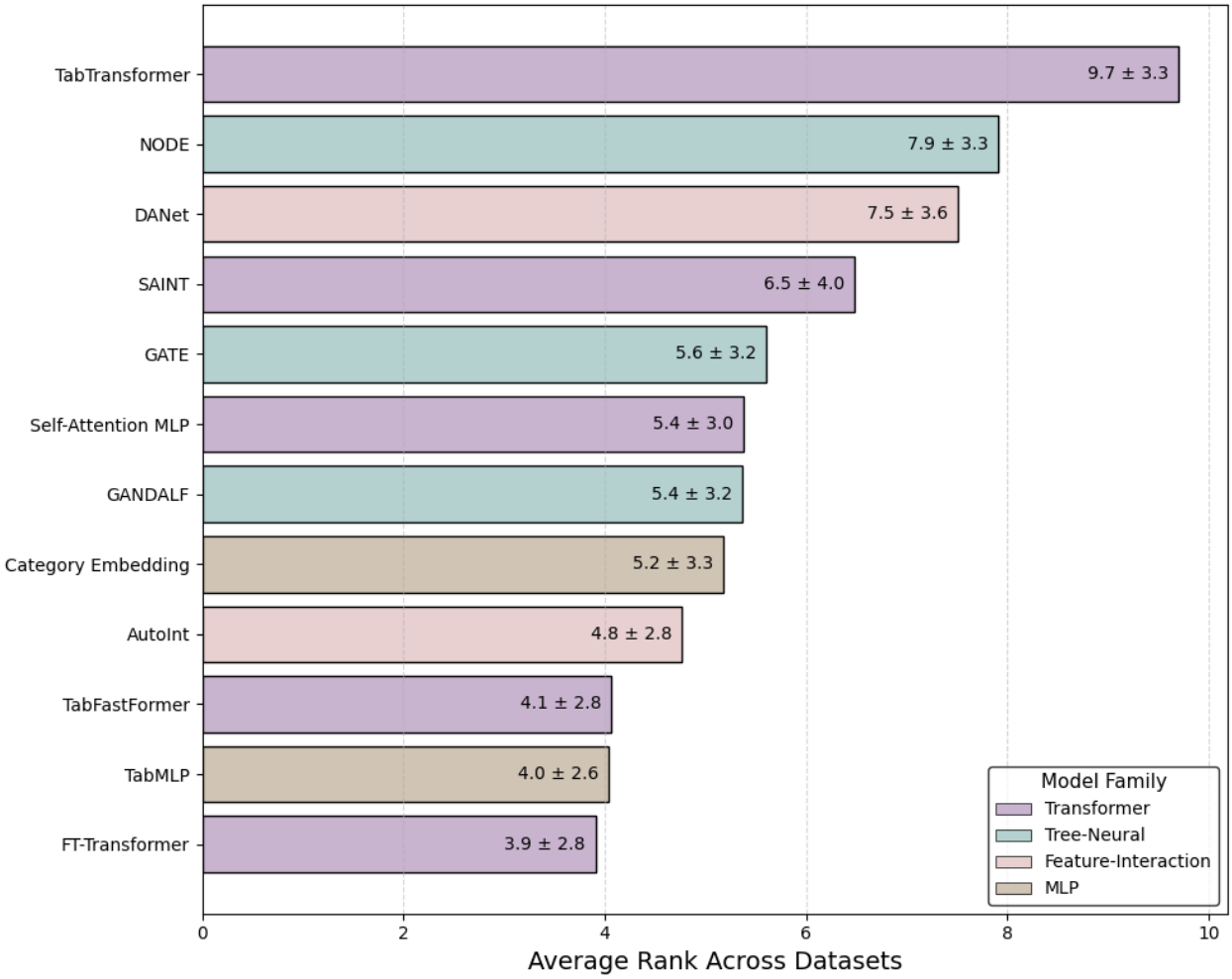
[a] Win rate = Times #1 / 182 datasets

**Figure 4.3 Model Performance Consistency: Top 3 Frequency vs. Mean Accuracy**




For each dataset, the accuracy of each model was determined, and the models were ranked 1 (highest accuracy) to 12 (lowest accuracy). Following McElfresh et al. [7] and Ye et al. [8], an "average rank" then was determined for each model by summing their ranks across datasets and dividing by 182. **Figure 4.3**, following this page, displays the average rank of each model across all datasets.

**Figure 4.3 Average Model Rank Across 182 Datasets (Shorter Is Better)**




The top three models cluster within a narrow band (average ranks 3.9 to 4.1). Their overlapping standard deviations indicate these models often traded positions across datasets. This suggests that any top-tier model could rank first on a large share of datasets. Performance differences are driven more by dataset characteristics than by algorithmic superiority. A clear gap separates the top tier from lower-performing models (Tab Transformer through SAINT, ranks 6.5 to 9.7).

## 4.2. Machine Learning and Deep Learning Performance Comparison

In an earlier benchmark report [9], 15 machine learning (ML) models were benchmarked using 159 datasets on a supervised binary classification task. The ML project and this DL benchmark used 158 datasets in common. **Figure 4.4**, following this page, displays the ranking of all 27 models' average accuracy on the 158 datasets.

**Figure 4.4 ML and DL Models Ranked by Overall Accuracy on 158 Datasets**

| Model | Type | Accuracy |
|---|---|---|
| XGBoost | ML | 86.02% |
| Random Forest | ML | 85.78% |
| LightGBM | ML | 85.74% |
| MLP | ML | 85.47% |
| Extra Trees | ML | 85.39% |
| SVC | ML | 84.98% |
| GBM | ML | 84.88% |
| TabFastFormer | DL | 83.48% |
| TabMLP | DL | 83.46% |
| Decision Tree | ML | 83.04% |
| Category Embedding | DL | 83.01% |
| KNN | ML | 82.99% |
| LogReg | ML | 82.89% |
| AutoInt | DL | 82.68% |

| Model | Type | Accuracy |
|---|---|---|
| Linear SVC | ML | 82.57% |
| FT-Transformer | DL | 82.45% |
| Self-Attention MLP | DL | 81.90% |
| SGD | ML | 81.77% |
| LDA | ML | 81.26% |
| QDA | ML | 80.98% |
| GANDALF | DL | 80.96% |
| GATE | DL | 78.33% |
| SAINT | DL | 77.90% |
| NODE | DL | 76.42% |
| DANet | DL | 75.48% |
| Naive Bayes | ML | 72.93% |
| TabTransformer | DL | 70.72% |

The top seven models are all ML, and the first DL model (FT-Transformer) appears at rank 6. For the 158 datasets and 27 models, ML models outperform the best DL models on tabular data.

**Figure 4.5**, on the following page, ranks all 27 models by their frequency of achieving the highest accuracy on individual datasets, providing a measure of dominance rather than average performance. Unlike mean accuracy, which can be elevated by consistently moderate results, win rate identifies which models are most often the best choice on any given dataset.

ML dominates the top of the rankings. The top five positions are all ML models, with XGBoost winning on 24 datasets (15.2% win rate), nearly double the next best model. Other observations include the following:

- XGBoost leads all 27 models with a 15.2% win rate, finishing first on 24 of 158 datasets — nearly double the next best model and confirming its dominance on tabular binary classification.
- The top five positions are held entirely by ML models, consistent with the widely reported finding that traditional ML outperforms DL on tabular data.

**Figure 4.5 ML and DL Model Win Rate Rankings Across 158 Shared Datasets**

| Rank | Model | Type | Times #1 | Times Top 3 | Win Rate % |
|---|---|---|---|---|---|
| 1 | XGBoost | ML | 24 | 66 | 15.2 |
| 2 | MLP | ML | 15 | 29 | 9.5 |
| 3 | Extra Trees | ML | 15 | 43 | 9.5 |
| 4 | LightGBM | ML | 13 | 48 | 8.2 |
| 5 | Random Forest | ML | 11 | 48 | 7.0 |
| 6 | FT-Transformer | DL | 11 | 32 | 7.0 |
| 7 | TabMLP | DL | 9 | 16 | 5.7 |
| 8 | AutoInt | DL | 9 | 19 | 5.7 |
| 9 | TabFastFormer | DL | 8 | 18 | 5.1 |
| 10 | SVM (RBF) | ML | 7 | 24 | 4.4 |
| 11 | Gradient Boosting | ML | 5 | 22 | 3.2 |
| 12 | Category Embedding | DL | 5 | 11 | 3.2 |
| 13 | SAINT | DL | 4 | 12 | 2.5 |
| 14 | Self-Attention MLP | DL | 3 | 8 | 1.9 |
| 15 | GANDALF | DL | 3 | 12 | 1.9 |
| 16 | Decision Tree | ML | 3 | 8 | 1.9 |
| 17 | KNN | ML | 2 | 8 | 1.3 |
| 18 | GATE | DL | 2 | 9 | 1.3 |
| 19 | Gaussian NB | ML | 2 | 2 | 1.3 |
| 20 | QDA | ML | 2 | 7 | 1.3 |
| 21 | Logistic Regression | ML | 2 | 8 | 1.3 |
| 22 | DANet | DL | 1 | 7 | 0.6 |
| 23 | Linear SVM | ML | 1 | 9 | 0.6 |
| 24 | TabTransformer | DL | 1 | 4 | 0.6 |
| 25 | LDA | ML | 0 | 1 | 0.0 |
| 26 | NODE | DL | 0 | 1 | 0.0 |
| 27 | SGD Classifier | ML | 0 | 2 | 0.0 |

- FT-Transformer is the strongest DL model, ranking 6th overall with an 11% win rate that matches Random Forest, outperforming seven ML models including SVM, Gradient Boosting, Decision Tree, and all linear classifiers.
- Three more DL models, TabMLP, AutoInt, and TabFastFormer, rank 7th through 9th, ahead of most ML models, suggesting the performance gap between the two classes of models is concentrated at the top rather than uniform across all models.
- Underperformance is not exclusive to DL. LDA, SGD Classifier, and Gaussian NB all finish at or near the bottom, alongside NODE and TabTransformer, confirming that within-class model selection matters as much as the ML versus DL choice itself.

- TabTransformer is the weakest DL model with only one first-place finish and a 0.6% win rate, a result that stands in sharp contrast to stronger Transformer-based models in the same family such as FT-Transformer and TabFastFormer.

TabTransformer ranks last across every metric evaluated in this benchmark, including accuracy, win rate, average rank, and Brier score. Its attention mechanism operates only on categorical feature embeddings, leaving numeric features unmodeled by attention entirely, which limits its ability to capture the feature interactions that drive performance on most tabular datasets. Section 4.6 examines this further, including a targeted hyperparameter tuning experiment.

## 4.3. Statistical Significance Testing

Statistical testing confirmed that the performance differences observed across the 12 models are real and not due to chance. The Friedman test[6] found at least one model performs significantly differently from others ($\chi^2 = 424.22$, df = 11, $p < 0.001$). The subsequent Nemenyi post-hoc test [7]evaluated all 66 pairwise model comparisons, finding that 40 (60.6%) show statistically significant differences.

The results find two distinct performance groups, shown in **Figure 4.5**. Group A has 11 models whose accuracy differences are not statistically distinguishable from one another. Practitioners can choose among them without concern for meaningful accuracy loss.

**Figure 4.5 Statistical Grouping from Nemenyi Post-Hoc Test**

| Group | Models | Mean Accuracy Range |
|---|---|---|
| A | TabFastFormer, TabMLP, Category Embedding, AutoInt, FT-Transformer, Self-Attention MLP, GANDALF, GATE, SAINT, NODE, DANet | 0.780–0.832 |
| B | TabTransformer | 0.727 |

TabTransformer stands alone in Group B as the only model significantly worse than the field. This isolation is confirmed across three independent measures: lowest win rate (4.9%), highest average rank (9.7), and sole membership in the lower statistical tier. DANet, while technically in Group A, is statistically distinguishable from several top performers and sits closer to the boundary than its group membership implies.

## 4.4. Prediction Calibration: Brier Score Analysis

Accuracy and F1-weighted measure whether a model classifies correctly, but they say nothing about how confident the model was in its predictions. A model that predicts 51% probability for every correct classification and 49% for every incorrect one will have the same accuracy as a model that predicts 99% and 1% respectively, yet these two models behave very differently in practice.

[6] A statistical test that compares multiple models across many datasets to determine whether their performance differences are meaningful rather than due to chance.

[7] A follow-up test that identifies which pairs of models differ significantly after an overall difference has been detected.

The Brier score measures the accuracy of probabilistic predictions by calculating the mean squared difference between a model's predicted probabilities and actual binary outcomes. A Brier score of 0 represents perfect probability predictions and 0.25 is the score of a naive model that always predicts 0.5. The Brier score penalizes overconfident wrong predictions and rewards well-calibrated probability estimates. This makes it a more complete measure of model quality in applications where predicted probabilities are used directly for decision-making.

For each of the 22 complexity measures, we computed a single correlation with mean accuracy across all 182 datasets. Results are presented in **Figure 4.6**.

**Figure 4.6. Brier Score and Accuracy Rankings Across 12 DL Models**

| Rank | Model | Mean Brier | Brier Std | Brier Rank | Mean Accuracy | Acc. Rank | Rank Differ -ence | N Data-sets |
|---|---|---|---|---|---|---|---|---|
| **1** | TabMLP | 0.1080 | 0.0726 | 1 | 0.8373 | 2 | -1 | 182 |
| **2** | Category Embedding | 0.1098 | 0.0816 | 2 | 0.8363 | 3 | -1 | 182 |
| **3** | FT-Transformer | 0.1103 | 0.0823 | 3 | 0.8322 | 5 | -2 | 179 |
| **4** | TabFastFormer | 0.1108 | 0.0736 | 4 | 0.8376 | 1 | 3 | 179 |
| **5** | AutoInt | 0.1134 | 0.0739 | 5 | 0.8339 | 4 | 1 | 181 |
| **6** | Self-Attention MLP | 0.1164 | 0.0781 | 6 | 0.8234 | 6 | 0 | 180 |
| **7** | GANDALF | 0.1227 | 0.0801 | 7 | 0.8188 | 7 | 0 | 182 |
| **8** | GATE | 0.1312 | 0.0855 | 8 | 0.7960 | 8 | 0 | 182 |
| **9** | SAINT | 0.1416 | 0.0945 | 9 | 0.7816 | 9 | 0 | 179 |
| **10** | DANet | 0.1447 | 0.1083 | 10 | 0.7711 | 11 | -1 | 182 |
| **11** | NODE | 0.1771 | 0.0759 | 11 | 0.7795 | 10 | 1 | 182 |
| **12** | TabTransformer | 0.1796 | 0.1054 | 12 | 0.7272 | 12 | 0 | 182 |

Brier score rankings are highly consistent with accuracy rankings, with 10 of 12 models showing a rank difference of 0 or |1| between the two metrics. This alignment confirms that models performing well on accuracy also tend to produce well-calibrated probability estimates, and that the two metrics largely agree on model quality across this benchmark.

TabMLP achieves the best Brier score (0.1080) despite ranking second on accuracy, while FT-Transformer ranks third on Brier but fifth on accuracy, suggesting it produces better-calibrated probabilities than its accuracy rank alone implies. TabFastFormer shows the largest divergence, ranking first on accuracy but fourth on Brier, indicating its predictions are slightly less well-calibrated relative to its classification performance. At the bottom, TabTransformer scores worst on both metrics, reinforcing its position as the weakest model in the benchmark by every measure evaluated.

The strong overall alignment between Brier score and accuracy rankings simplifies model selection. Practitioners do not face a significant tradeoff between predictive accuracy and probability calibration when choosing among these 12 DL models. The accuracy-based rankings

presented throughout this report can be considered dependable guides to calibration quality as well.

## 4.5. Memory Efficiency and Stability

Beyond accuracy, models also were evaluated for memory efficiency. Model memory footprint is a critical factor in real-world machine learning deployment, directly impacting cloud compute costs and system scalability. For each model, on every dataset, we measured the maximum amount of VRAM (Video Random Access Memory, or GPU memory) used during the GPU training cycles.

However, absolute maximum VRAM usage is not a reliable comparison point, as it is influenced by many external factors. These include the dataset's size, data types, and sparsity, as well as the operating system, library implementations, hardware configuration, available VRAM, and background processes or memory management overheads.

To provide a fairer assessment, two comparative measures were used:

- ***Multiple of Minimum VRAM***: For each dataset, the ratio of a model's VRAM usage to the minimum observed was calculated and then averaged across all 182 datasets. This shows how much more memory a model typically requires compared to the most efficient option. Values close to 1.0 indicate near-optimal efficiency, while larger values point to models that consistently demand more resources.
- ***Stability Factor***: For each dataset, overhead was defined as the difference between a model's VRAM usage and the minimum observed for that dataset. A model's Stability Factor is the standard deviation of these overhead values across all 182 datasets. A low value means predictable memory use, while a high value indicates large swings in demand and greater risk in deployment.

The results in **Figure 4.7**, on the following page, reveal a trade-off between efficiency (cost) and stability (risk). The most memory-efficient models were Category Embedding, GANDALF, TabTransformer, and DANet. While these four were not the top accuracy performers, they are competitive. Category Embedding offers had a strong accuracy (3rd overall) with the lowest memory footprint of any model, making it the standout choice for resource-constrained deployments. Category Embedding, GANDALF, and TabTransformer had a good combination of reasonable accuracy, low memory footprint, and predictable resource demands.

Tree-neural hybrids are consistently inefficient. GANDALF at 3.38× (good), GATE at 16×, and NODE at 661× (catastrophic). This family has the highest risk profile: large swings, high overhead, and extreme worst-case behavior.

Stability is a separate dimension from efficiency. A model can be moderately inefficient but highly predictable (TabTransformer: 4.14x multiple, 1.16 stability factor), or highly inefficient but consistent in that inefficiency (DANet: 5.82x, 1.06 stability factor). AutoInt, TabMLP, Self-Attention MLP, SAINT, and FT-Transformer all show high values on both dimensions, indicating they are simultaneously memory-hungry and unpredictable, which is a challenging combination for deployment.

**Figure 4.7 Model VRAM Efficiency and Stability Factors (most efficient first)**

| Model | Multiple of Minimum VRAM | Stability Factor | Family |
|---|---|---|---|
| Category Embedding | 2.84 | 1.00 | MLP-based |
| GANDALF | 3.38 | 2.05 | Tree-neural hybrid |
| TabTransformer | 4.14 | 1.16 | Transformer-based |
| DANet | 5.82 | 1.06 | Feature Interaction/Attention |
| TabFastFormer | 12.95 | 5.70 | Transformer-based |
| GATE | 16.41 | 5.87 | Tree-neural hybrid |
| AutoInt | 18.38 | 20.10 | Feature Interaction/Attention |
| TabMLP | 37.27 | 30.73 | MLP-based |
| Self-Attention MLP | 27.19 | 23.36 | Transformer-based |
| SAINT | 55.44 | 32.74 | Transformer-based |
| FT-Transformer | 61.46 | 33.20 | Transformer-based |
| NODE | 661.51 | 54.46 | Tree-neural hybrid |

## 4.6 Evaluating TabTransformer's Sensitivity to Hyperparameter Tuning

Five of the datasets used in this benchmark also appear in the original TabTransformer study by Huang et al. [10] The accuracies reported in that paper are substantially higher than those observed in this benchmark, as shown in **Figure 4.8**.

**Figure 4.8 TabTransformer Performance Gap: Huang et al. vs. This Benchmark**

| Dataset | Haung Accuracy | Benchmark Accuracy | Difference |
|---|---|---|---|
| albert | 75.7% | 63.95% | (11.75)% |
| jasmine | 85.3 | 73.87 | (11.43) |
| philippine | 83.4 | 68.21 | (15.19) |
| shrutime | 85.6 | 81.90 | (3.70) |
| spambase | 98.5 | 78.83 | (19.67) |

Several factors explain these gaps:

- Dataset-specific hyperparameter tuning in Huang et al. versus none here
- Tailored preprocessing versus a uniform preprocessing here
- Potentially different train/validation/test splits, including cross-validation
- More extensive training configurations by Huang, including learning rate scheduling and warm-up periods
- Implementation differences between Huang's original codebase and the PyTorch Tabular implementation used in this study.

Notably, this benchmark's TabTransformer accuracy also falls below Huang's MLP baseline (not shown here) on all five shared datasets. The results show how strongly the absence of tuning, tailored preprocessing, and optimized training configurations affects this model's performance.

To investigate whether tuning could close this gap, six hyperparameters were tuned across 141 of the 182 datasets:

- Number of attention heads
- Transformer blocks
- Embedding dimension
- Attention dropout
- Feed-forward dropout
- Learning rate.

**Figure 4.9** summarizes the results. Tuning improved accuracy on 61% of datasets, but the pattern of improvement is revealing. The largest gains occurred on small, simple, predominantly numeric datasets where TabTransformer's default configuration had produced near-random or inverted predictions — recoveries from catastrophic failure rather than genuine architectural optimization. On larger, more complex datasets with categorical features, tuning produced little or no benefit, suggesting that TabTransformer's performance ceiling on those datasets is architectural rather than a hyperparameter calibration issue.

**Figure 4.9. Summary of TabTransformer Hyperparameter Tuning Outcomes**

| Metric | Value |
|---|---|
| Datasets tuned | 141 |
| Datasets improved (> +0.5 pp) | 86 (61.0%) |
| Datasets with meaningful improvement (> +2.0 pp) | 78 (55.3%) |
| Datasets with no change | 41 (29.1%) |
| Datasets degraded (< -0.5 pp) | 14 (9.9%) |
| Median improvement, all datasets (pp) | +4.40 |
| Mean improvement, all datasets (pp) | +8.46 |
| Largest improvement | Raisin (+74.44 pp, baseline: 16.7%, tuned: 91.1%) |
| Largest decline [a] | parity5 (-50.00 pp, baseline: 75.0%, tuned: 25.0%) |
| Strongest positive correlates | T3 ($r$ = +0.27), T2 ($r$ = +0.24) |
| Strongest negative correlates | F1v ($r$ = -0.43), L1 ($r$ = -0.38), LSC ($r$ = -0.38) |
| Original accuracy vs improvement | $r$ = -0.46 (lower accuracy → larger tuning gain) |

[a] The five datasets with the largest declines all had 100 or fewer records

## 5. Dataset Complexity Assessment

Dataset complexity was measured using 22 indicators from the Lorena et al. framework. [2] The analysis identifies which structural characteristics influence classification difficulty. Results

show that geometric and neighborhood complexity drive classification difficulty more than dataset size or class imbalance.

This section is organized as follows:

5.1 Dataset Complexity Landscape

5.2 Dataset Complexity Impact on Model Performance

5.3 Class Imbalance Resolution

5.4 Feature Importance Validation

5.5 Computational Properties of the clsCoef Complexity Measure.

### 5.1. Dataset Complexity Landscape

This project evaluates model performance across 182 binary classification datasets drawn from public repositories. Understanding dataset complexity helps explain variation in model performance.

Dataset complexity was measured using 22 indicators defined by Lorena et al. (2019) [2], grouped into six categories:

- **Feature-based**: discriminative power of individual attributes
- **Linearity**: decision boundary complexity
- **Neighborhood**: local class overlap and boundary definition
- **Network**: graph-theoretic connectivity patterns
- **Dimensionality**: effects of high-dimensional spaces
- **Class imbalance**: representation of minority classes.

These measures are described in **Appendix A** to this report. The measures capture geometric, topological, and distributional properties that affect how easily algorithms can separate classes. An overall complexity score also was computed for each dataset as the arithmetic mean of all 22 standardized measures. All measures were calculated using the Python library *problexity*.

The overall complexity score across the 182 datasets has a median of 0.41 and an IQR of 0.14, indicating a mostly moderate-difficulty collection with relatively consistent complexity. **Figure 5.1** displays the full distribution.

**Figure 5.1 Distribution of Benchmark Datasets by Complexity Score**

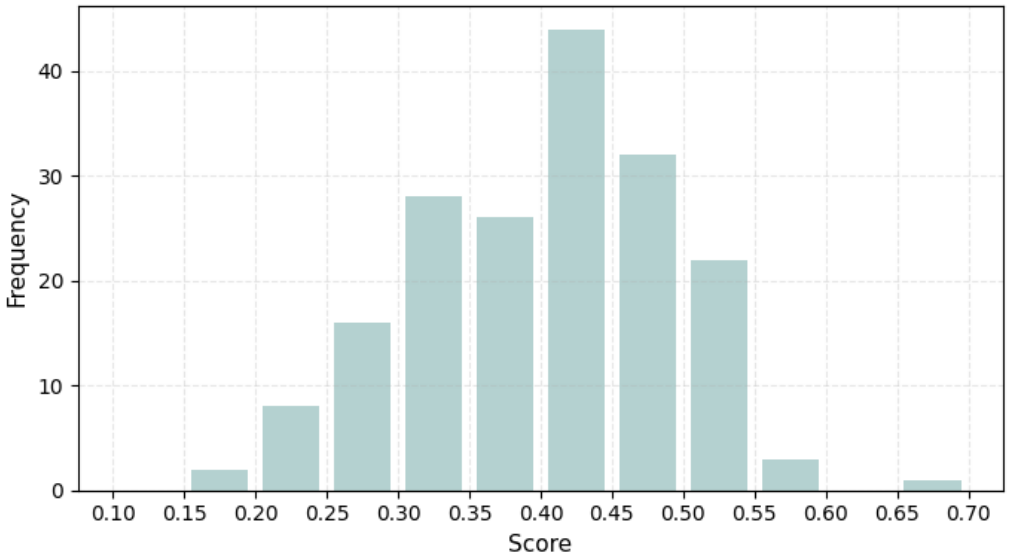
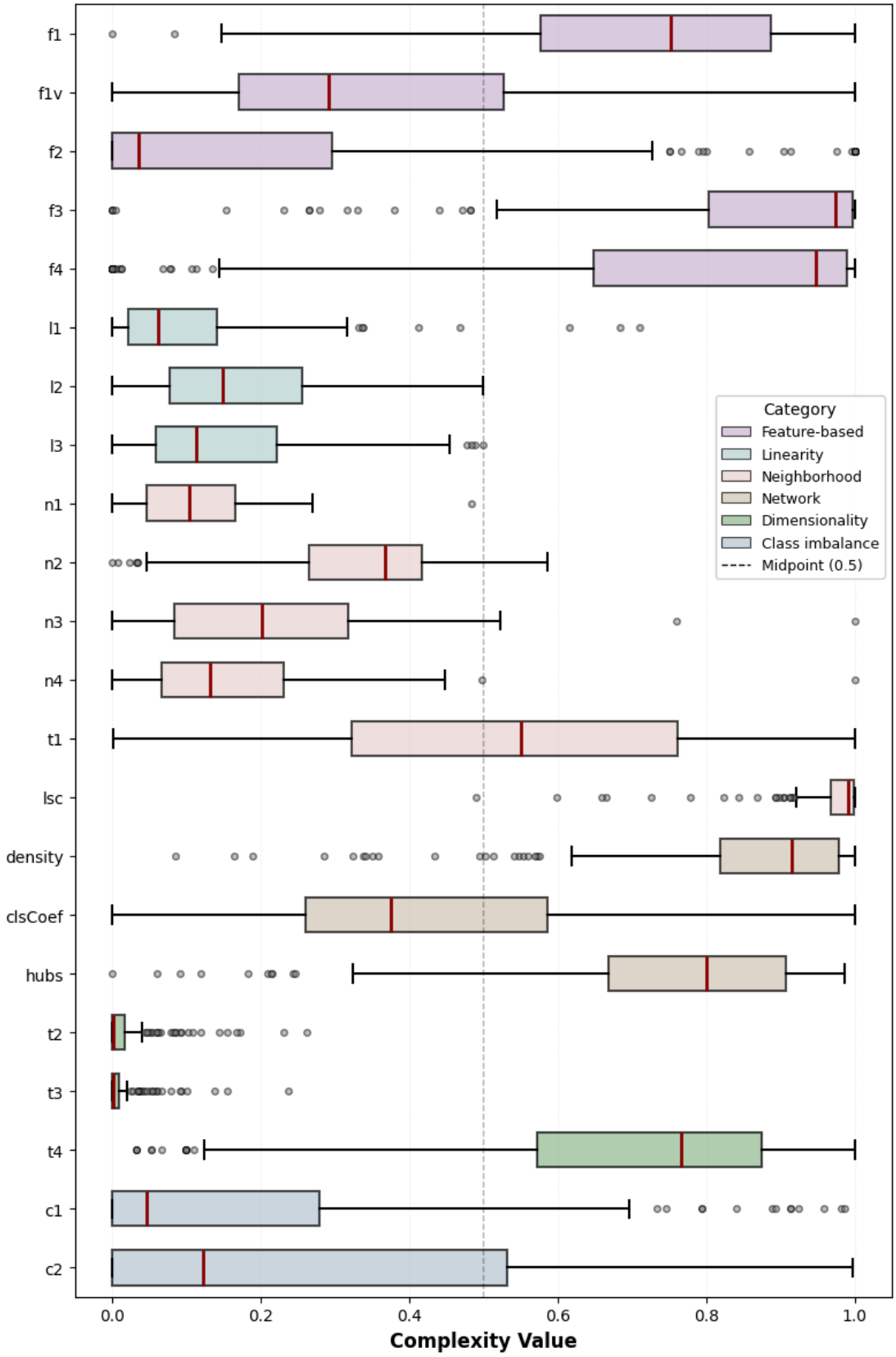



**Exhibit 5.1**, on the following page, summarizes the distributions of all 22 complexity measures across the 182 datasets, grouped by category. There are substantial differences in geometric, topological, and distributional characteristics.

The overall complexity score across the 182 datasets has a median of 0.41 and an IQR of 0.14, indicating a mostly moderate-difficulty collection with relatively consistent complexity. Although the datasets span the full range of structural difficulty, they come from curated repositories and lack many real-world irregularities such as missing values, mislabeled targets, and concept drift. They provide a controlled comparison environment, but limit generalizing about model performance, and are not a substitute for raw operational data.

Complexity varies substantially across the six categories. Feature-based and network measures are the dominant challenges, as the only two categories exceeding the overall complexity median (category medians of 0.70 and 0.60 respectively), reflecting that individual features often lack discriminative power and that same-class neighborhood structure creates difficult separation problems. Neighborhood complexity sits near the overall median, with LSC showing the highest median of all 22 measures (0.991), indicating consistently narrow and irregular decision boundaries across the collection. Linearity, dimensionality, and class imbalance all fall well below the overall median. Class imbalance shows the lowest median of all categories (0.085) but the highest variability (std = 0.304), reflecting a collection that is predominantly balanced but contains a minority of severely imbalanced datasets.

**Exhibit 5.1 Distribution of Complexity Measures Across 182 Datasets**



## 5.2. Dataset Complexity Impact on Model Performance

Earlier Section 5.1 profiled the complexity profile of the 182 datasets. This section examines which complexity dimensions most strongly influenced classification difficulty by correlating each with model accuracy. These relationships address the second research question: what structural traits make datasets difficult to classify?

Pearson correlation coefficients were calculated by pairing each complexity measure with model accuracy across all 2,172 model-dataset pairs (12 models × 182 datasets, minus 12 model-dataset pairs that could not complete). This model-agnostic approach pools results across all 12 models to identify dataset complexity traits that consistently influence classification difficulty. The results show that geometric and topological complexity, particularly neighborhood overlap and decision boundary non-linearity, dominate classification difficulty, while traditionally emphasized factors like class imbalance and dataset size show surprisingly weak and positive relationships with performance.

### *1. Correlations Between Complexity Measures and Model Accuracy*

To better understand which dataset characteristics are associated with overall model performance difficulty, we aggregated performance across all models, using the mean accuracy of all models on each dataset. That is, we used just one accuracy observation per dataset. This avoids within-dataset model variations on each dataset that are present.

**Figure 5.2** provides the correlation of each dataset complexity measure with model prediction accuracy. [8]All correlations were statistically significant ($p < 0.0001$), confirming that each complexity measure relates to classification performance, though the strength and direction of these relationships vary substantially. Five measures have **strong negative** correlations ($r < -0.70$), two have **stronger positive** correlations ($r > 0.50$), while the remaining fourteen showed **weaker** associations.

**Figure 5.2 Complexity Measure Correlations with Model Accuracy (all *p* <0.0001, N = 2,172) (Sorted in Descending Absolute Value)**

| Measure | Correlation | Strong |
|---|---|---|
| N1 | (0.866) | **Yes** |
| N3 | (0.833) | **Yes** |
| N4 | (0.804) | **Yes** |
| L2 | (0.752) | **Yes** |
| L3 | (0.705) | **Yes** |
| T1 | (0.625) | **Yes** |
| C1 | 0.558 | **Yes** |
| F1v | (0.550) | **Yes** |
| C2 | 0.545 | **Yes** |

| Measure | Correlation | Strong |
|---|---|---|
| T3 | (0.346) | **Yes** |
| F3 | (0.299) | **Yes** |
| LSC | (0.276) | **Yes** |
| F1 | (0.272) | **Yes** |
| L1 | (0.256) | **Yes** |
| F4 | (0.253) | **Yes** |
| Density | (0.249) | **Yes** |
| clsCoef | (0.239) | **Yes** |
| F2 | (0.237) | **Yes** |

[8] Correlation values in Figure 5.2 are computed using one row per dataset. For each dataset, the target performance metric (e.g., Accuracy) was averaged across all 12 models before correlating with the 22 Lorena complexity measures.

| Measure | Correlation | Strong |
|---|---|---|
| N2 | (0.492) | **Yes** |
| T2 | (0.372) | **Yes** |

| Measure | Correlation | Strong |
|---|---|---|
| Hubs | (0.170) | **Yes** |
| T4 | 0.029 | **No** |

The strongest predictors of classification difficulty are measures of class overlap near the decision boundary: specifically, how many data points sit close to points from the opposite class. When the two classes mix together in the feature space, all models struggle regardless of architecture. Linearity measures, which capture how curved or irregular the boundary between classes must be, are the next strongest predictors. By contrast, class imbalance and network structure measures show weak or counterintuitive relationships with accuracy, making them less reliable indicators of dataset difficulty than boundary geometry.

Neighborhood complexity were the strongest predictor of classification difficulty, with three measures ranking highest: N1 ($r$ = -0.87), N3 ($r$ = -0.83), and N4 ($r$ = -0.80). This confirms that datasets with extensive class overlap near decision boundaries consistently challenge models across all learning algorithms. Linearity measures followed closely, with L2 ($r$ = -0.75) and L3 ($r$ = -0.70) demonstrating that non-linear decision boundaries substantially increase classification difficulty.

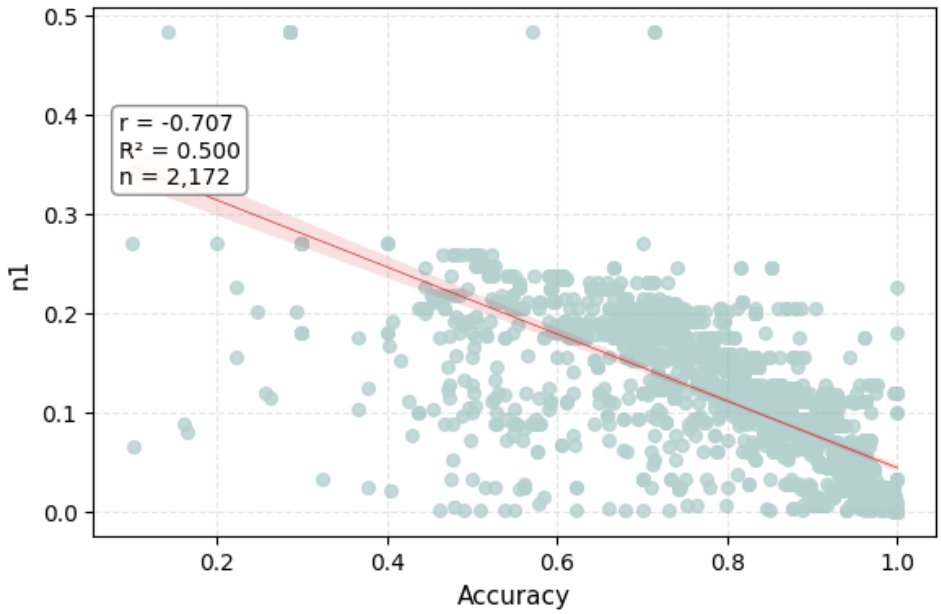
Feature-based measures showed moderate correlations (F1v: r = -0.55, F1: r = -0.27), indicating that weak individual feature discriminability contributes meaningfully to dataset difficulty, though less powerful than geometric properties. Class imbalance measures showed a counterintuitive pattern: both C1 ($r$ = +0.56) and C2 ($r$ = +0.55) showed positive correlations with accuracy, indicating that higher imbalance is associated with better performance rather than worse. This counterintuitive finding contradicts conventional wisdom that imbalanced datasets are harder to classify and require deeper investigation (and addressed further in Section 5.3).

Hubs was the only measure with a weak correlation ($|r| < 0.200$) with performance. Hubs contributes relatively little to distinguishing dataset difficulty.

**Figure 5.3**, on the following page, illustrates the relationship between neighborhood complexity measure N1 (fraction of borderline points) and model accuracy.[9] N1 demonstrated a moderate negative correlation with model accuracy ($r$ = -0.707, $p < 0.001$), explaining only 50% ($R^2$ = 0.500) of the variance in performance. The remaining 50% reflects variation in how models handle neighborhood complexity.

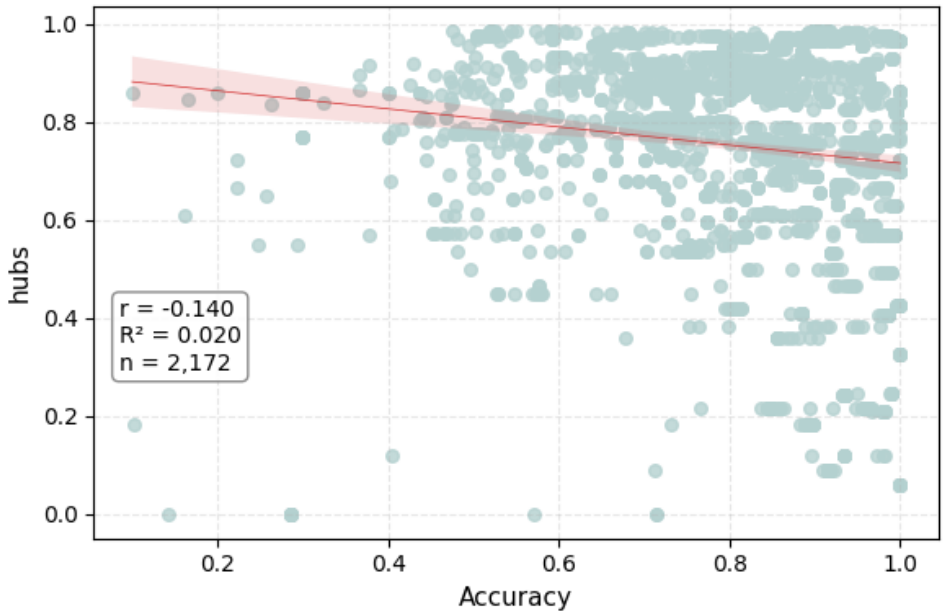
[9] The scatter plot uses all 2,172 model–dataset pairs. Each point represents a single model's performance on a specific dataset, paired with that dataset's complexity value. This produces a model-level correlation that reflects both a dataset's difficulty and within-dataset variation in model performance. This differs from the dataset-level correlations earlier in Figure 5.2.

**Figure 5.3 N1 (Neighborhood Complexity) vs. Model Accuracy**




In contrast to N1, the network measure Hubs (class connectivity via instance hubs) demonstrated the statistically weakest correlation of all 22 measures ($r$ = -0.140, $R^2$ = 0.020). **Figure 5.4** displays the comparison. A Hub is a data point that is a nearest neighbor to a significantly larger number of other points than the average. This measure captures the structural homogeneity of the dataset. A low score for this concentration suggests a stable, centralized structure (low complexity), while a high score suggests a dispersed, fragile structure (high complexity). The nearly horizontal regression line and negligible $R^2$ confirm that Hubs explains virtually none of the accuracy variation.

**Figure 5.4 Hubs (Class Connectivity Via Instance Hubs) vs. Model Accuracy**



The Hubs scatter plot reveals substantial horizontal spread at specific Hubs values, most notably at Hubs ≈ 0.982 where a series of 48 dots represent four datasets. On these datasets, models achieved highly divergent accuracies ranging from 0.355 to 0.967. This pattern confirms that while Hubs itself does not determine difficulty, underlying geometric characteristics cause some models to struggle while others perform effectively on the same problem.

### *2. Validation Through Easy vs. Hard Dataset Comparison*

To confirm the correlation findings, datasets were stratified by model consensus. Twenty-six datasets where the mean accuracy of all models was ≥95% were classified as "easy," while the 58 151 datasets with a mean accuracy of not greater than 65% were classified as "hard." For each complexity measure, mean values were calculated separately for the easy and hard groups. The hard mean was divided by the easy mean to show how many times higher the measure was in hard datasets (e.g., 10.6× means hard datasets averaged 10.6x higher). This enables comparison of complexity characteristics between trivially easy and genuinely difficult problems.

Dataset size showed no consistent pattern distinguishing easy from hard datasets. Median record counts differed significantly (18,459 easy vs. 2,229 hard, or ≈ 88% gap). Feature counts presented mixed results: hard datasets *averaged* more features (≈39 vs. 33) yet had a lower *median* (11 vs. 18). These mixed results indicate that scale, whether measured by records or features, did not determine classification difficulty.

Dataset size did not explain difficulty, but complexity measure comparisons revealed a consistent pattern: geometric complexity (boundary shape and linearity) and topological complexity (neighborhood structure and connectivity) drive dataset difficulty. **Figure 5.5** shows that 12 measures showed 3-18 fold increases in hard datasets, all statistically significant ($p \leq 0.05$).

**Figure 5.5 Complexity Measures with Largest Correlation Differences Between Easy and Hard Datasets ($r_{hard}$ / $r_{easy}$)**

| Measure | Easy | Hard | Multiple | Complexity Category | *p*-value |
|---|---|---|---|---|---|
| L3 | 0.014 | 0.253 | 17.9x | Linearity | 0.000 |
| L2 | 0.018 | 0.289 | 16.1x | Linearity | 0.000 |
| T2 | 0.003 | 0.034 | 11.8x | Dimensionality | 0.002 |
| N4 | 0.024 | 0.272 | 11.3x | Neighborhood | 0.000 |
| T3 | 0.002 | 0.020 | 10.6x | Dimensionality | 0.000 |
| N3 | 0.038 | 0.367 | 9.7x | Neighborhood | 0.000 |
| N1 | 0.020 | 0.188 | 9.6x | Neighborhood | 0.000 |
| N1 | 0.020 | 0.188 | 9.6x | Neighborhood | 0.000 |
| F2 | 0.064 | 0.321 | 5.1x | Feature-based | 0.000 |
| F1v | 0.144 | 0.540 | 3.8x | Feature-based | 0.000 |
| T1 | 0.252 | 0.755 | 3.0x | Neighborhood | 0.000 |
| L1 | 0.047 | 0.139 | 3.0x | Linearity | 0.000 |

Several Lorena complexity measures show large, multi-fold increases from Easy to Hard datasets, especially those tied to linearity, dimensionality, and neighborhood structure. These measures demonstrate that Hard datasets show far more class overlap, more irregular boundaries, and more ambiguous local neighborhoods, producing increases ranging from 3× to nearly 18×. The overall complexity score[10] (not shown in figure) is 40% higher in hard datasets (0.332 to 0.460, $p < 0.001$), confirming that multiple complexity dimensions compound classification difficulty.

However, several measures showed only minimal differences between Easy and Hard datasets. A second group, including F3, clsCoef, F1, Density, Hubs, LSC, and T4, shows minimal change between Easy and Hard datasets (1.0×–1.4×). These weak shifts indicate that large portions of the Lorena framework do not meaningfully discriminate dataset difficulty in this benchmark. T4, which measures the ratio of the PCA dimension to the original dimension, was the only measure that showed no correlation with accuracy.

### *3. Model Family Sensitivity to Dataset Complexity*

To assess whether complexity dimensions affect model families differently, Pearson correlations were calculated between each of the 22 complexity measures and accuracy for each of the eight model families separately. Results, including a heatmap, are presented in **Appendix D**.

#### *Overall Pattern*

- Nearly all 22 complexity measures show negative correlations with accuracy, confirming that as dataset complexity increases, model performance decreases across all four architecture families.
- The strength of correlations varies widely, from very strong (N1 at -0.866) to essentially zero (T4 at 0.029), indicating that some complexity dimensions are far more consequential than others.

#### *Dominant Measures: Neighborhood Complexity*

- N1, N3, and N4 are the three strongest predictors of accuracy across all families, with overall correlations of -0.866, -0.833, and -0.804 respectively, confirming that decision boundary complexity and local class overlap are the primary drivers of classification difficulty for DL models.
- Linearity measures L2 and L3 also show strong consistent correlations (-0.752 and -0.705), indicating that datasets requiring highly non-linear decision boundaries present substantial challenges regardless of architecture.
- T1, the fraction of maximum covering spheres, adds further evidence that topological boundary complexity matters, with an overall correlation of -0.625.

[10] The overall complexity "score" produced by Python's problexity is the arithmetic mean of the 22 complexity measures.

### *Class Imbalance: The Positive Correlation*

- C1 and C2 are the only measures with positive correlations (0.558 and 0.545), meaning greater class imbalance is associated with higher accuracy rather than lower accuracy, consistent with the Class Imbalance Paradox discussed in Section 5.3.

### *Family Differences*

- The Transformer and MLP families show the strongest correlations with neighborhood measures, suggesting these architectures are more sensitive to boundary complexity than Tree-neural and Feature-Interaction families.
- Tree-neural hybrids show the weakest correlations overall across most measures, suggesting they are somewhat more robust to complexity variation, though still meaningfully affected.
- All four families respond to complexity measures in the same direction and similar magnitude, indicating that dataset difficulty affects DL architectures broadly rather than selectively.

### *Weak or Negligible Measures*

- T4 shows a near-zero overall correlation (0.029), confirming it has no meaningful relationship with DL model accuracy across these 182 datasets.
- Hubs, density, clsCoef, and F2 all show weak correlations below -0.25, suggesting network structure measures contribute little independent predictive value for accuracy beyond what neighborhood measures already capture.

## 5.3 Class Imbalance Resolution

An imbalanced binary classification dataset has significantly more examples of one class than the other. Class imbalance measures (C1, C2) showed positive correlations with model performance ($r$ = +0.56, +0.55, $p < 0.001$). However, comprehensive correlation analysis revealed this relationship to be spurious. **Figure 5.6**, on the following page, shows that C1 has negative correlations with 15 of 17 other complexity measures with statistically significant correlations ($p < 0.05$). This pattern indicates that highly imbalanced datasets in this collection also showed lower geometric and topological complexity.

The positive correlation between class imbalance and higher model performance is a spurious relationship driven by the confounding factor of geometric simplicity. In this dataset collection, highly imbalanced cases (high C1) consistently exhibit lower structural and neighborhood complexity, meaning the rare minority class is geometrically distinct and easier for models to isolate. Conversely, the balanced datasets in this suite tend to have overlapping boundaries and higher "geometric friction," which makes the classification task mathematically more difficult despite the equal distribution of examples. Therefore, the models are not performing better because of the imbalance, but rather because the imbalanced datasets in this study happen to have much simpler and more separable data structures.

**Figure 5.6 C1's Correlations with Each of the 22 Complexity Measures (in descending |*r*|)**

| Measure | Correlation (*r*) | *p*_Value | Significant? |
|---|---|---|---|
| **C2** | 0.9681 | 0.9681 | Yes |
| **L2** | (0.5566) | 0.5566 | Yes |
| **N1** | (0.5302) | 0.5302 | Yes |
| **N3** | (0.4826) | 0.4826 | Yes |
| **T1** | (0.4800) | 0.4800 | Yes |
| **N4** | (0.4781) | 0.4781 | Yes |
| **L3** | (0.4580) | 0.4580 | Yes |
| **Density** | (0.4050) | 0.4050 | Yes |
| **Hubs** | (0.3714) | 0.3714 | Yes |
| **LSC** | (0.3609) | 0.3609 | Yes |
| **N2** | (0.3243) | 0.3243 | Yes |
| **F3** | (0.2410) | 0.2410 | Yes |
| **F1** | 0.2295 | 0.2295 | Yes |
| **F2** | (0.2199) | 0.2199 | Yes |
| **F4** | (0.2092) | 0.2092 | Yes |
| **F1v** | (0.1882) | 0.1882 | Yes |
| **clsCoef** | (0.1753) | 0.1753 | Yes |
| **T2** | (0.1386) | 0.1386 | No |
| **T3** | (0.1257) | 0.1257 | No |
| **T4** | (0.0484) | 0.0484 | No |
| **L1** | 0.0423 | 0.0423 | No |

In a second analysis of class imbalance, both accuracy and F1 increased as class ratios became more extreme, with the most imbalanced datasets (95/5 majority/minority) showing the highest average performance. **Figure 5.7** summarizes these results.

**Figure 5.7 Accuracy and F1 Scores by Imbalance Ratio**

| Ratio | Bin Count | Accuracy | Accuracy Std. | F1Score | F1Score Std. | Bin Percent |
|---|---|---|---|---|---|---|
| **50/50** | 132 | 0.831 | 0.149 | 0.806 | 0.184 | 6.1% |
| **60/40** | 204 | 0.780 | 0.130 | 0.738 | 0.165 | 9.4% |
| **70/30** | 384 | 0.796 | 0.133 | 0.761 | 0.161 | 17.7% |
| **80/20** | 336 | 0.798 | 0.142 | 0.771 | 0.167 | 15.5% |
| **90/10** | 216 | 0.855 | 0.144 | 0.834 | 0.163 | 9.9% |
| **95/5 or worse** | 900 | 0.804 | 0.180 | 0.789 | 0.203 | 41.4% |
| | 2,172 | 0.806 | 0.158 | 0.782 | 0.184 | 100.0% |

The relationship between class imbalance and model performance is neither linear nor intuitive. Balanced datasets (50/50) perform well at 83.1% accuracy, but the 90/10 bin achieves the highest accuracy of any group at 85.5%, suggesting that when the minority class is rare but well-defined, models can still perform effectively. The most severely imbalanced bin (95/5 or worse) contains the largest share of observations and shows near-average accuracy (80.4%), but its standard deviation is the highest of any group on both accuracy (0.180) and F1-weighted (0.203). The variability produces unpredictable rather than uniformly poor results.

### 5.4 Feature Importance Validation

To validate the correlation results, two feature importance methods were applied: Random Forest (RF) importance and Linear Regression (LR) standardized coefficients. RF captures nonlinear relationships and interaction effects, while LR highlights interpretable linear effects. Both methods independently assessed the influence of all 22 complexity measures on classification accuracy.

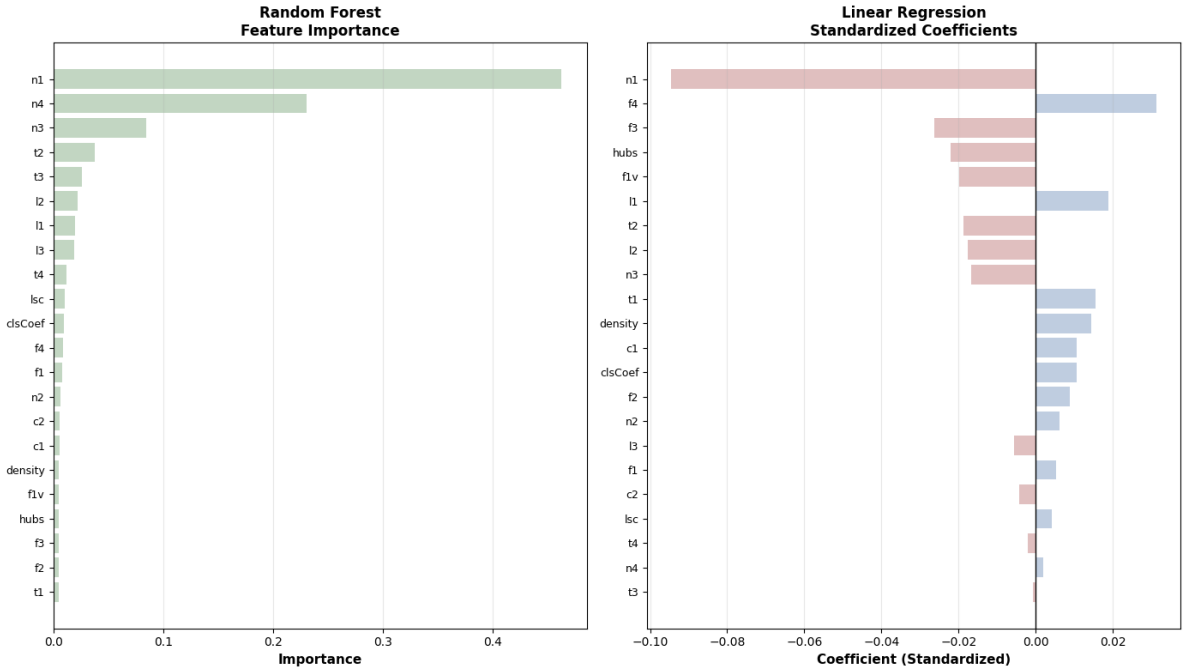
The consensus top seven measures identified as important (by average rank) are:

1. N1 (RF rank: #1, LR rank: #1)
2. T2 (RF rank: #4, LR rank: #7)
3. N3 (RF rank: #3, LR rank: #9)
4. L1 (RF rank: #7, LR rank: #6)
5. L2 (RF rank: #6, LR rank: #8)
6. F4 (RF rank: #12, LR rank: #2)
7. F3 (RF rank: #20, LR rank: #3).

**Figure 5.8**, on the following page, shows strong consensus on the most influential measures. The consensus top seven measures span four of the six Lorena et al. categories are: Neighborhood, Dimensionality, Linearity, and Feature-based. This suggests that classification accuracy on tabular datasets is shaped by multiple independent structural properties rather than any single dominant category. Network and Class Imbalance measures are notably absent from the top seven, consistent with the earlier finding that these categories present limited obstacles in this benchmark collection.

N1 is the standout finding, ranking first by both RF and LR. As the fraction of borderline points in a minimum spanning tree, N1 directly captures decision boundary complexity: the degree to which class regions interleave at their edges. Its dual top ranking across two independent methods with different assumptions makes it the most robust predictor of accuracy in this benchmark.

**Figure 5.8 Dataset Complexity Measures Feature Importance**




T2, the average number of samples per feature, ranks consistently in both methods (RF: 4, LR: 7). Its importance reflects that dataset sparsity, or having too few samples compared to feature count, constrains what any model can learn regardless of architecture.

N3 and L1/L2 show moderate divergence. N3 (kNN error rate) ranks 3rd by RF but 9th by LR, suggesting it captures nonlinear boundary effects that linear models underweight. L1 and L2 (linearity measures) are more consistent across methods, showing their relationship with accuracy is relatively linear in nature.

## 5.5 Computational Properties of the clsCoef Complexity Measure

Calculating the 22 Lorena et al. complexity measures revealed an unexpected runtime anomaly. The clsCoef measure alone accounts for approximately 81% of total problexity runtime, with a mean of 495 seconds per dataset compared to a combined mean of just 5.4 seconds for the other 21 measures. One dataset required over five hours. This contradicts the literature, which identifies N1 (mean 43 seconds) and T1 (mean 10 seconds) as the primary computational bottlenecks.

The root cause is the interaction between clsCoef's fixed epsilon threshold and the Gower distance metric on datasets with binarized or categorical features, which produces explosively large local neighborhoods. The algorithm's inner loop scales as $k_i^2$ per point and is implemented in unvectorized Python, causing runtime to grow catastrophically as neighborhood density increases. Spearman correlations confirm this empirically: k_i_sum_sq achieves r = 0.926 with

clsCoef runtime ($p < 0.001$). Practitioners running the Lorena et al. measures at scale should budget for clsCoef separately and monitor runtime on datasets with high proportions of binary or categorical features

# 6. Model Performance by Dataset Characteristics

Section 5 established that geometric and topological complexity, particularly neighborhood overlap and decision boundary non-linearity, determine classification difficulty across the benchmark collection. This section examines how different models respond to these six complexity categories, identifying which models maintain performance under specific challenging conditions and which ones show fundamental limitations.

This section is organized as follows:

6.1 Model Performance by Complexity Category

6.2 Model Performance by Dataset Size.

### *6.1 Model Performance by Complexity Category*

All 12 models were assessed across the six complexity categories using the top 25% most complex datasets in each category (46 to 48 datasets per category). Performance was classified as Strong (top 5), Moderate (middle 5), or Weak (bottom 5) based on mean accuracy. This tests each model's robustness under genuinely challenging conditions rather than overall benchmark performance.

The results, shown in **Figure 6.1** and **Figure 6.2**, following this page, reveal a clear hierarchy. AutoInt, Category Embedding, and TabMLP earned Strong ratings across all six categories, while FT-Transformer and TabFastFormer earned Strong ratings in five of six. These five models are reliable generalists, maintaining performance regardless of complexity type.

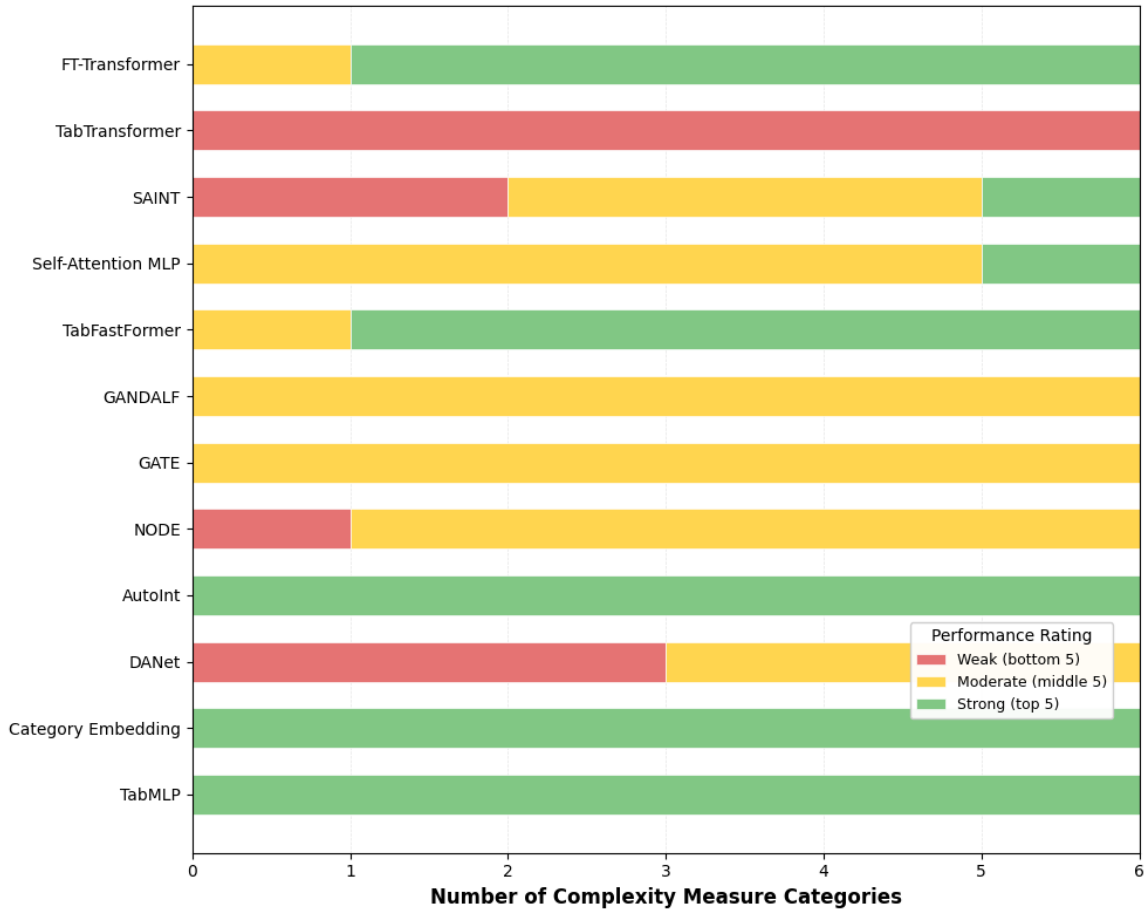
GANDALF and GATE were consistently Moderate across all categories. DANet and TabTransformer were the weakest, with TabTransformer earning Weak ratings in all six categories, consistent with its last-place finish on every other metric in this benchmark.

A notable structural finding is that the Transformer family spans the entire performance spectrum from best (FT-Transformer) to worst (TabTransformer), making family membership a poor predictor of robustness. Architectural choices within a family matter more than the family itself.

**Figure 6.1 Model Strength on Most Complex Datasets (sorted by model family, then model)**

| Model | Feature | Linearity | Neighbor-hood | Network | Dimen. | Class Imb. | Strong Count | Weak Count |
|---|---|---|---|---|---|---|---|---|
| FT-Transformer | Strong | Strong | Strong | Strong | Moderate | Strong | 5 | 0 |
| TabTransformer | Weak | Weak | Weak | Weak | Weak | Weak | 0 | **6** |
| SAINT | Moderate | Moderate | Moderate | Weak | Strong | Weak | 1 | 2 |
| Self-Attention MLP | Moderate | Moderate | Moderate | Moderate | Moderate | Strong | 1 | 0 |
| TabFastFormer | Strong | Strong | Strong | Strong | Strong | Moderate | 5 | 0 |
| GANDALF | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | 0 | 0 |
| GATE | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | 0 | 0 |
| NODE | Moderate | Moderate | Weak | Moderate | Moderate | Moderate | 0 | 1 |
| AutoInt | Strong | Strong | Strong | Strong | Strong | Strong | **6** | 0 |
| DANet | Weak | Weak | Moderate | Moderate | Weak | Moderate | 0 | 3 |
| Category Embedding | Strong | Strong | Strong | Strong | Strong | Strong | **6** | 0 |
| TabMLP | Strong | Strong | Strong | Strong | Strong | Strong | **6** | 0 |

**Figure 6.2 How Each Model Performs Under High Complexity Conditions**



### *6.2 Model Performance by Dataset Size*

This subsection evaluates how dataset size affects accuracy and stability, highlighting consistent performance patterns and notable exceptions. To evaluate model scalability, the 182 datasets were partitioned into three size buckets: small (≤2,000 records), medium (2,001–20,000), and large (>20,000). Each of the 12 models was assessed on its average performance within each bucket using mean accuracy and coefficient of variation (CV%) as metrics. This design tests whether models maintain accuracy and consistency as dataset size increases, or whether performance degrades on larger, more complex conditions. Buckets varied in sample count, feature dimensionality, and average complexity score, allowing for a robust comparison across scale. **Figure 6.3** presents a summary of the results.

**Figure 6.3 Model Performance Summary by Dataset Number of Records**

| Records | Datasets | Complex-ity Score | Mean Accuracy | Top 3 Models | Bottom 3 Models |
|---|---|---|---|---|---|
| <=500 | 60 | 0.387 | 0.739 | TabFastFormer, TabMLP, Category Embedding | GATE, DANet, TabTransformer |
| 2,001-20,000 | 64 | 0.401 | 0.827 | FT-Transformer, TabMLP, AutoInt | SAINT, NODE, TabTransformer |
| >20,000 | 58 | 0.422 | 0.853 | FT-Transformer, GATE, GANDALF | NODE, TabTransformer, SAINT |
| | **182** | **0.393** | **0.806** | **TabFastFormer, TabMLP, Category Embedding** | **NODE, DANet, TabTransformer** |

Across dataset sizes, the top models maintain their relative ordering. MLP-based and Feature Interaction/Attention models (TabMLP, Category Embedding, AutoInt) remain consistently strong across all sizes, while TabFastFormer and FT-Transformer rise to the top primarily because they scale especially well with larger datasets.

The largest performance spread occurs in the Small category, where the gap between the top model (TabFastFormer at 81.0%) and the bottom (TabTransformer at 59.8%) is 21.2 percentage points. This gap narrows considerably in Large datasets (86.9% vs 83.8%), suggesting that small dataset performance is the strongest test of model robustness and efficiency.

For small datasets specifically, MLP-based models and TabFastFormer offer the best combination of accuracy and consistency. For large datasets, performance differences narrow considerably and model choice matters less.

## 7. Model Performance versus Throughput

This section examines the computational costs of the 12 models, specifically predictions per second. It identifies which models are both fast and accurate, and which trade speed for predictive strength.

This section is organized as follows:

7.1 Model Throughput (Predictions per Second)

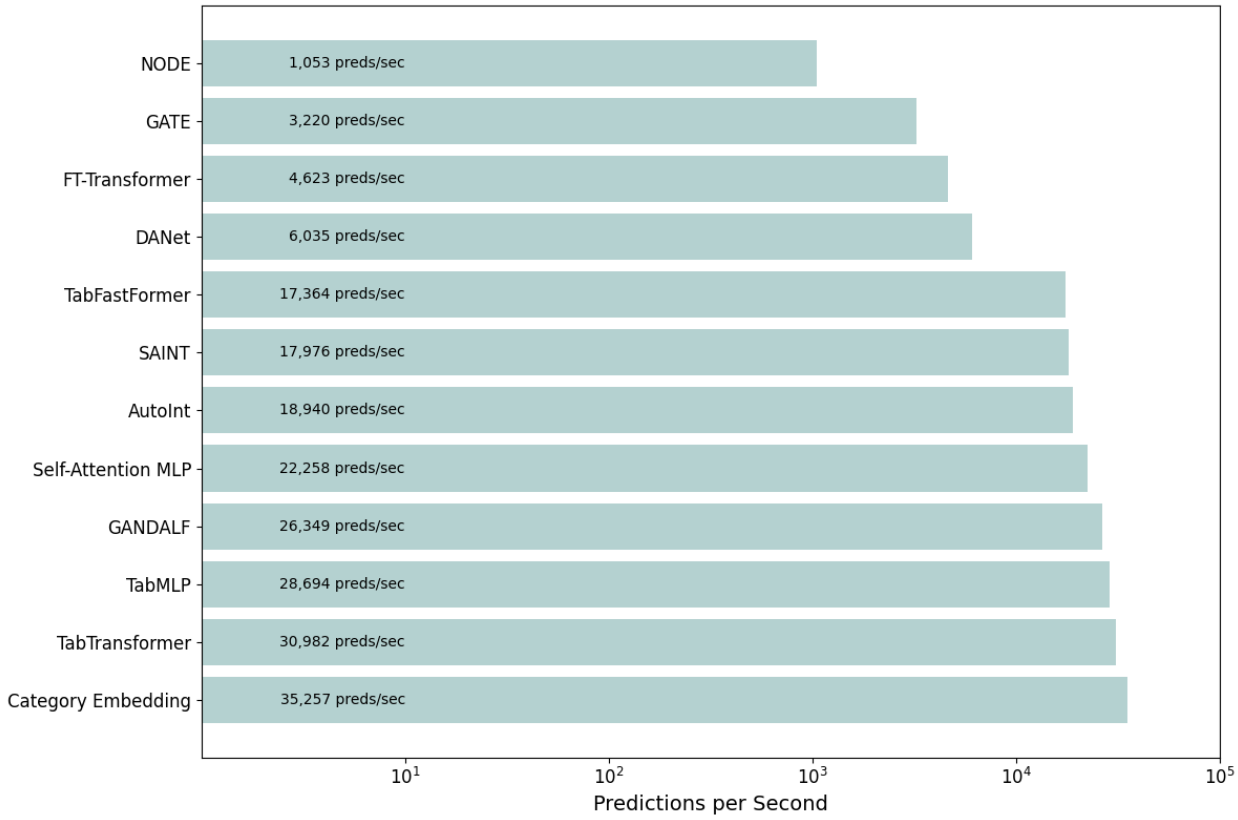
7.2 Performance-Throughput Trade-offs

7.3 Integrated Performance Score.

## 7.1 Model Throughput (Predictions per Second)

Prediction speed is a critical factor for real-time and high-volume deployment scenarios. Each model's baseline throughput was measured as its average predictions per second. While actual speed depends on compute resources, the relative differences between models are key, revealing intrinsic computational efficiency. **Figure 7.1** displays these results, with significant variability immediately apparent on the logarithmic scale.

**Figure 7.1 Model Predictions per Second ($\log_{10}$)**




Throughput varied by as much as approximately 33× across models. MLP-based models are the fastest family overall, with both Category Embedding and TabMLP in the top three, consistent with their architectural simplicity. Transformer-based models cluster in the mid-to-high range with the notable exception of FT-Transformer at 4,623, which is surprisingly slow relative to its Transformer family peers.

Feature Interaction/Attention models split sharply, with AutoInt nearly three times faster than DANet. The Tree-neural hybrid family shows the widest internal spread of any family, with NODE at the slowest of all models (1,053) while GANDALF reaches 26,349. GATE is in the lower tier at 3,220, making this family unpredictable from a throughput standpoint.

## 7.2 Performance-Throughput Trade-offs

This subsection examines the trade-offs between predictive accuracy and prediction speed. While accuracy is often the primary goal, throughput may determine model choice in real-time

or resource-limited deployments. The goal is to identify which models deliver strong performance without excessive computational cost, and which ones force a compromise.

The fundamental trade-off between model accuracy and computational efficiency is illustrated in **Figure 7.2**., on the following page. **Figure 7.3** further details this relationship, using average model ranking and model family groupings to identify the specific clusters that represent the effective compromises.

Figure 7.3 reveals a segmentation of models into four groups:

- **Upper left (high speed, high rank — best):** Category Embedding, TabMLP, and GANDALF occupy the optimal zone, combining fast throughput with strong average rankings. These are the most practically attractive models overall.
- **Upper right (high speed, poor rank):** TabTransformer achieves high throughput but ranks last on accuracy, making its speed advantage largely irrelevant given its poor performance.
- **Lower left (slow, high rank):** FT-Transformer delivers strong accuracy rankings but at significantly lower throughput, presenting a clear speed-accuracy tradeoff for practitioners.
- **Scattered middle and lower**: SAINT, AutoInt, DANet, GATE, and NODE occupy various mid-field positions with no compelling combination of speed and accuracy ranking.

The tradeoffs show that throughput cannot be treated as secondary to accuracy. Model differences in predictive strength and computational demands directly affect their suitability for deployment. Latency is critical for online systems, where slow predictions can break real-time requirements, while batch systems can tolerate longer runtimes. In production settings, throughput constraints may rule out otherwise strong models. These charts clarify which algorithms are practical when both speed and accuracy matter.

## 7.3 Integrated Performance Score

This subsection offers a different look at the benchmarking results by combining accuracy and throughput into a single, easily interpretable measure. Rather than evaluating accuracy and throughput separately, this integrated score offers a high-level view that simplifies communication and supports decision-making. The composite score weights accuracy at 60% and throughput at 40%, producing a weighted score for each model.

**Figure 7.2 Model Trade-offs: Performance and Prediction Speed (upper right is best)**

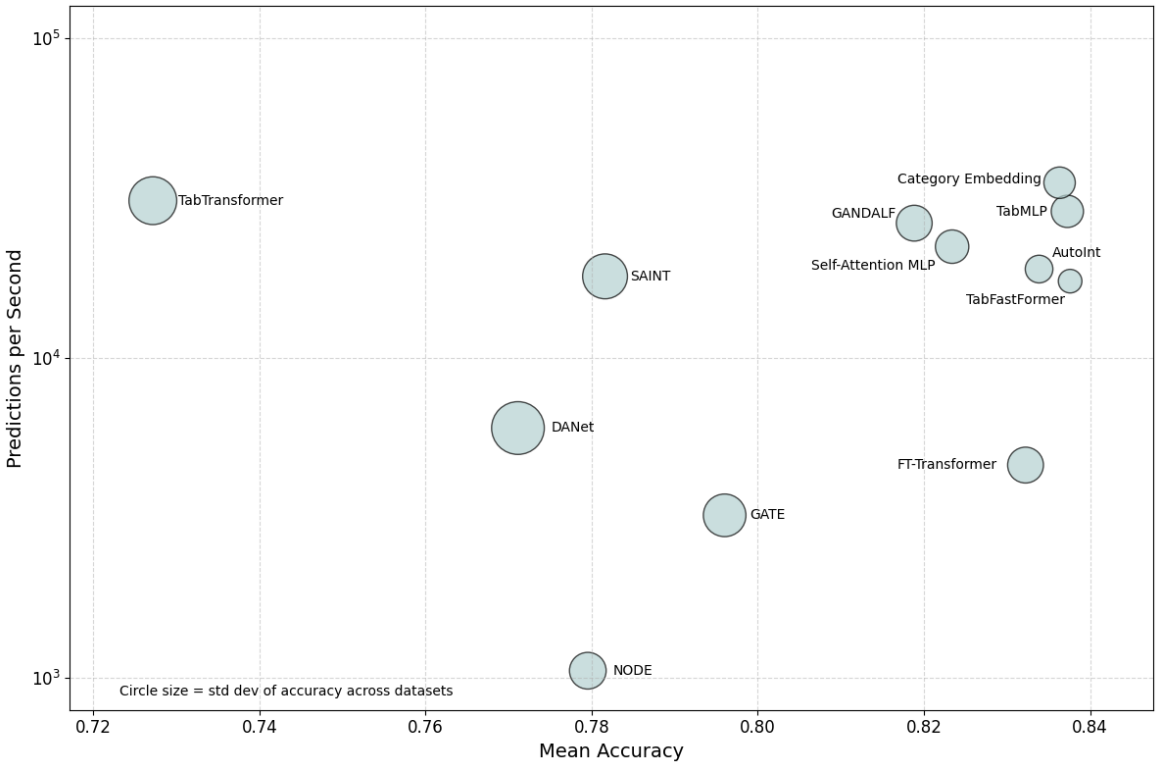



**Figure 7.3 Performance Rank vs. Prediction Speed by Model Family (upper left is best)**

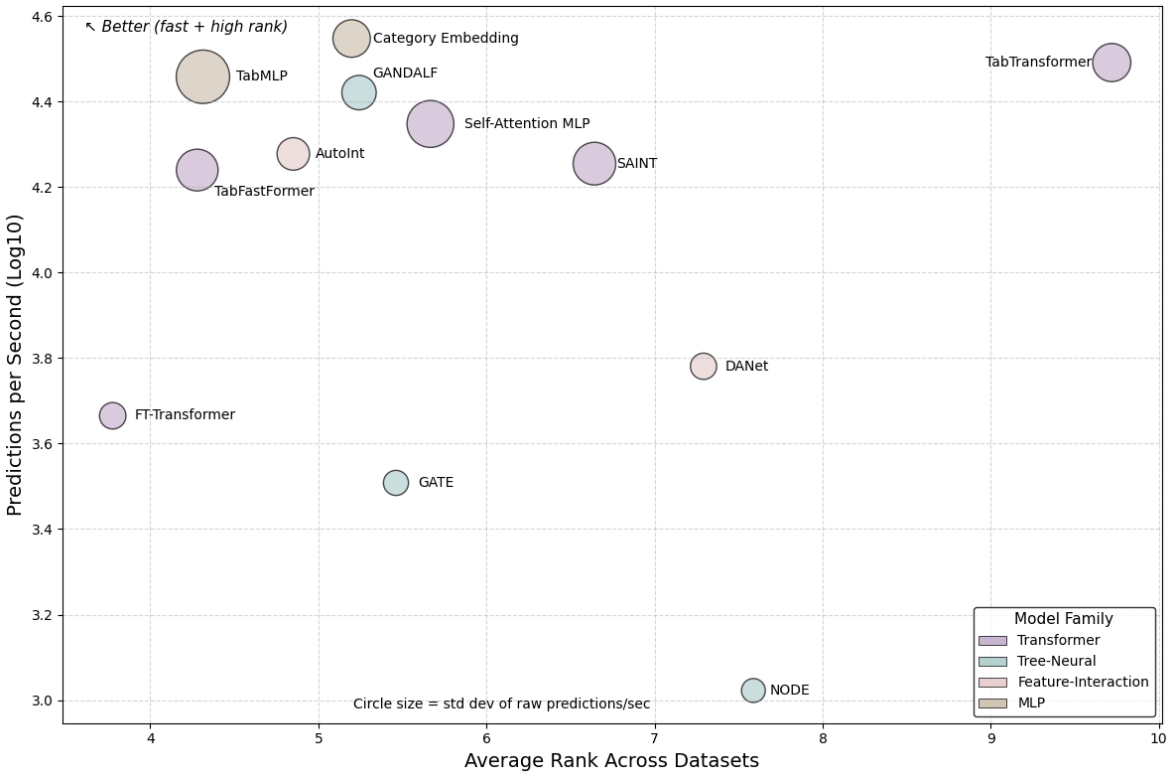


**Figure 7.4**, on the following page, shows each model's underlying normalized values for both metrics.[11] **Figure 7.5** presents the combined weighted score of both measures and indicates model family. This approach merges both metrics into a single interpretable "deployment" score, offering a means to explain and compare models.

The deployment score reshapes the landscape of model performance. Models with high accuracy but low throughput, such as FT-Transformer and Self-Attention MLP, shift downward when efficiency is factored in.

The integrated score highlights clear differences in how model families balance predictive performance with computational efficiency. CategoryEmbedding and TabMLP stand out with the highest composite scores, indicating that MLP-based architectures offer a strong combination of accuracy and fast inference. The MLP architectural simplicity translates to practical deployment advantages on tabular binary classification tasks.

Several transformer-inspired models, including TabFastFormer and Self-Attention MLP, also perform well, though others in the same family show more modest results. Tree-neural hybrids and feature-interaction models display a wider spread, with GANDALF and AutoInt achieving competitive scores while NODE, GATE, and DANet fall toward the lower end.

This integrated view supports more informed model selection. The 60/40 weighting is illustrative, not prescriptive. If speed is critical, adjusting the weights will elevate fast but less accurate models. If accuracy is paramount, the top tier models remain dominant. Presenting both normalized metrics and the weighted score side by side demonstrates how priorities shape model choices. The optimal balance depends on the use case: whether the goal is maximum accuracy, speed, or a compromise.

[11] Scores are normalized using min-max scaling to ensure comparability across models. Each value is rescaled to a 0–1 range based on the lowest and highest observed mean accuracy or throughput.

**Figure 7.4 Model Comparison: Normalized Accuracy vs. Throughput (longer is better)**

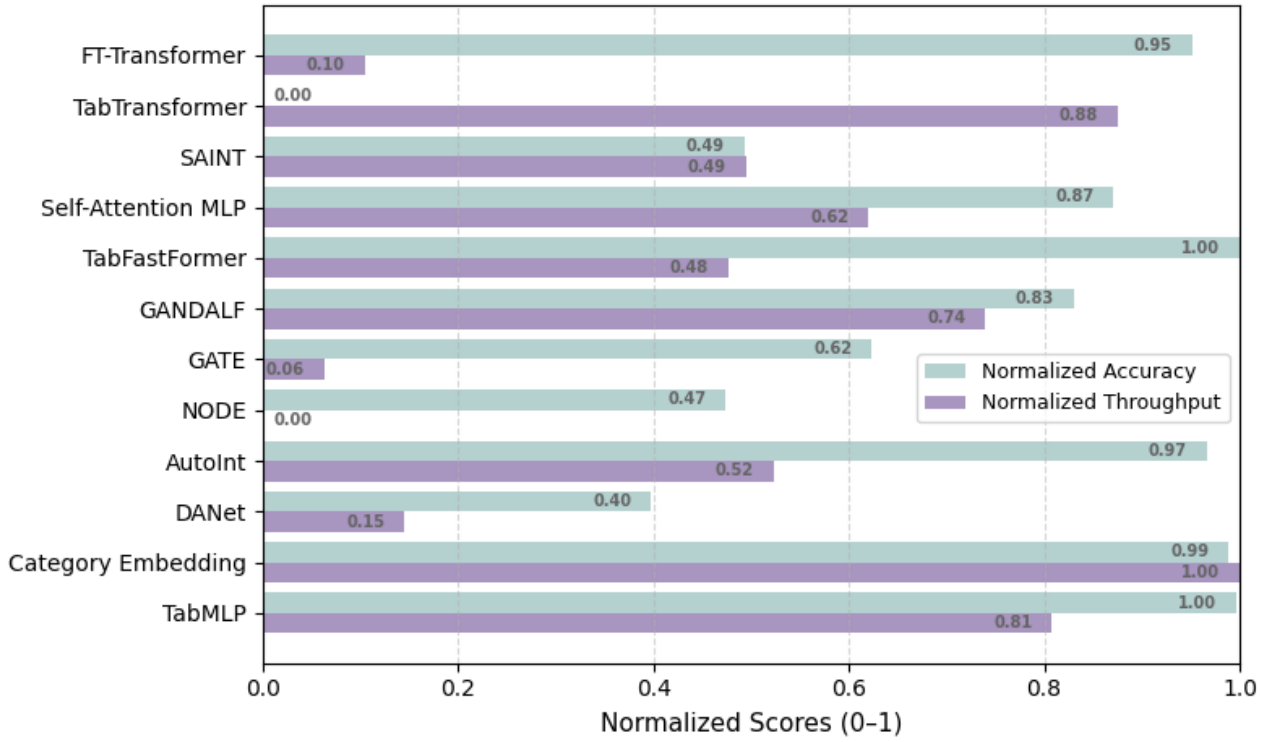
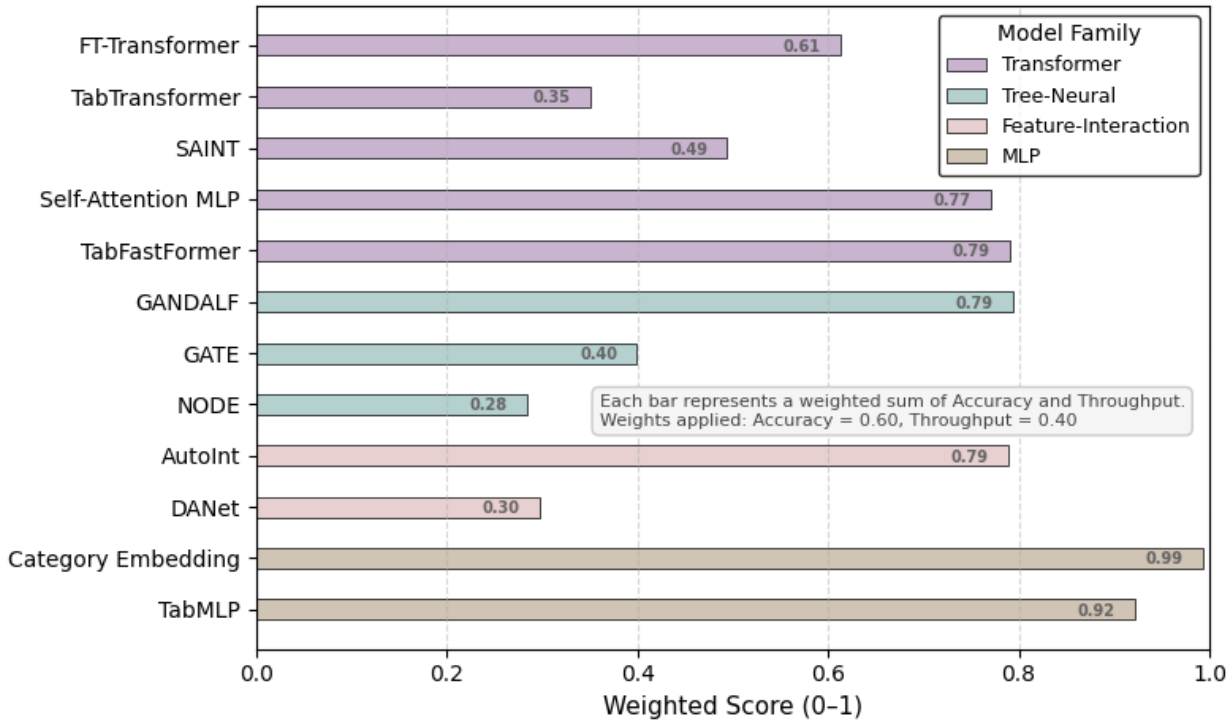



**Figure 7.5 Model Deployment Score Based on Combined Accuracy and Speed (longer is better)**



# 8. Conclusion

This study benchmarked 12 deep learning models across 182 tabular binary classification datasets, evaluating predictive performance, computational efficiency, probability calibration, and robustness under varying dataset complexity. The central finding is that architectural sophistication does not reliably translate to better performance or deployment advantage on tabular data.

For most tabular binary classification needs:

- Category Embedding and TabMLP are the strongest all-around choices, delivering both high accuracy and fast inference
- FT-Transformer is the best option when accuracy is the only priority and throughput constraints allow it
- NODE, DANet, and TabTransformer are difficult to recommend for production deployment regardless of the weighting scheme applied.

Eleven of the 12 models form a statistically equivalent top tier by formal testing, with mean accuracy ranging from 77.1% to 83.8%. While no single adjacent pair differs significantly, the full spread of 6.7 percentage points is not trivial in practice. Practitioners should weigh both the statistical grouping and the absolute accuracy range when selecting a model, alongside practical factors such as speed, memory, and cost. When accuracy and throughput are combined into a single deployment score, Category Embedding and TabMLP are the strongest all-around choices, delivering both high accuracy and fast inference.

TabTransformer is the sole outlier, ranking last on every metric evaluated including accuracy, win rate, Brier score, and average rank. These results are consistent with its architectural limitation of applying attention only to categorical features, so it may miss important patterns in numeric data.

Compared to 15 traditional ML models on 158 shared datasets, ML occupies the top five positions on accuracy, confirming that traditional ML remains stronger on tabular data. However, the gap is not uniform. The best DL models match mid-tier ML, and model selection within each approach matters as much as the choice between ML and DL itself.

Neighborhood complexity measures (N1, N3, N4) are the strongest predictors of classification difficulty, confirmed independently by two feature importance methods. Linearity measures follow closely. Class imbalance, contrary to common assumption, showed positive correlations with accuracy, explained by the fact that highly imbalanced datasets in this collection tended to have simpler geometric structure. Dataset geometry is a more reliable predictor of model difficulty than class distribution or dataset size alone.

A novel finding concerns the Lorena et al. complexity measures themselves. The clsCoef measure accounts for approximately 81% of total computation time across 182 datasets, directly contradicting the literature's identification of neighborhood and tree-based measures as the primary bottlenecks. This has practical implications for any researcher computing these measures at scale.

# 9. Limitations and Future Work

**Dataset Collection:** Based benchmarks on 182 tabular binary datasets from public repositories (OpenML, UCI, Kaggle, Other). Larger collections, private/proprietary datasets, and domain-specific data were not represented, nor were higher complexity datasets. *Future Work: Expand to larger and more diverse datasets, including domain-specific collections (healthcare, finance, commerce, education), to strengthen generalizability and confirm findings across application areas.*

**Hyperparameter Optimization:** Default hyperparameters (the settings that control how a model learns, chosen before training begins) were used for all 12 models. This choice ensures a fair, out-of-the-box comparison, but it does not reflect each model's best possible performance. The TabTransformer tuning experiment showed that default settings can produce extremely poor results on some datasets, and this risk applies to all models. *Future Work: Investigate whether systematic hyperparameter tuning changes model rankings or reduces performance gaps between top-tier models.*

**Statistical Methodology**: Correlations between complexity measures and accuracy were computed at the dataset level, using mean accuracy across all 12 models per dataset. This approach avoids the independence violation that would result from treating each model-dataset pair as a separate observation, but it averages over meaningful within-dataset variation in model performance. *Future Work: Explore weighted or hierarchical correlation methods that preserve model-level variation while maintaining statistical validity.*

**Feature Importance:** Random Forest and Logistic Regression were used to assess complexity measure importance. Both methods capture limited aspects of feature influence and can be misleading when relationships are nonlinear or involve interactions. *Future Work: Add complementary methods such as permutation importance and SHAP values to confirm the stability of importance rankings.*

**Robustness to Noisy Data:** The benchmark uses curated datasets that lack real-world irregularities such as extreme missingness, mislabeled targets, or corrupted features. Model rankings may differ when data quality is poor. *Future Work: Introduce controlled noise into a subset of datasets to assess which architectures are most resilient to real-world data quality issues.*

# Appendix A - Dataset Complexity Measures

This appendix provides definitions for the 22 dataset complexity measures used to characterize the 182 datasets in this study. The measures are grouped into the six categories defined in Lorena et al. (2019) [2]. An overview of each category is provided below:

- **Feature-based measures (F1, F1v, F2, F3, F4)**: Evaluate the ability of individual features, or combinations of features, to separate classes. Examples include the maximum Fisher's discriminant ratio and the volume of the overlapping region between classes.
- **Linearity measures (L1, L2, L3)**: Assess how well classes can be separated using linear boundaries, often by utilizing a Linear Support Vector Machine (SVM) and assessing its error rate or the distance of misclassified points from the decision boundary.
- **Neighborhood measures (N1, N2, N3, N4, T1, LSC)**: Focus on local distribution of instances, analyzing class overlap, borderline examples, and decision boundary complexity based on nearest neighbors.
- **Network measures (Density, clsCoef, Hubs)**: Convert the dataset into a graph structure to evaluate properties like dataset density and connectivity, providing insights into topological complexity.
- **Dimensionality measures (T2, T3, T4)**: Capture sparsity by relating the number of samples to number of features (dimensions), as high dimensionality can increase problem difficulty.
- **Class imbalance measures (C1, C2)**: Quantify imbalance in class sample size, a factor that can significantly affect classification difficulty.

The 22 measures defined in Lorena et al. [2] include the 12 originally developed and described by Ho and Basu [1]. Together, they provide evidence of whether dataset classes are well separated or heavily interleaved, an important determinant of classification accuracy. [1] To standardize interpretation, modifications were introduced [2] into the earlier definitions [1], so that all measures assume values within bounded intervals, with higher values consistently indicating greater complexity and lower values indicating lesser complexity.

For this project, values of all 22 measures were calculated for each of the 182 datasets. Calculating the measures excluded all non-numeric features. Non-numeric features were excluded and remaining numeric features were standardized using scikit-learn's StandardScaler to ensure comparability across measures.

Because of compute limits,[12] 70 datasets (of 182) with more than 15,000 records were stratified sampled to 15,000 instances while preserving class balance. After preprocessing, the *problexity* Python library was applied to each dataset's feature matrix and target label, generating a score for each measure.

The *problexity* module automatically inverts scores for measures where lower values originally indicated higher complexity. In general, values close to 0 represent lower complexity, while values close to 1 represent higher complexity. This normalization ensures consistent directional logic across all measures. The module also provides an overall dataset difficulty score, calculated as the arithmetic mean of the 22 individual measures.

The complexity measures are defined in six categories by [2], and subsequently rephrased by [11] and [12]. The tables that follow define each measure and interpret the complexity level it represents. For selected measures where definitions may not be immediately intuitive, formulas are included to provide clarity and replicability.

### Category 1. Feature-Based Measures

This group evaluates how effectively individual features, or combinations of features, can distinguish between classes in the dataset. These measures capture the separability of the data based on its raw attributes, without relying on model-specific behavior. A high number of discriminative features tends to simplify the dataset. **Figure C-1**, on the following page, provides a description of the five measures in this category.

[12] For datasets exceeding 15,000 records, a stratified random sample was drawn to prevent memory overflow during complexity calculations. This limitation reflects current inefficiencies in the problexity implementation, which does not efficiently leverage available multicore resources (28 cores in our environment).

**Figure C-1. Feature-Based Measures**

| Acronym | Name and Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **F1** | **Maximum Fisher's Discriminant Ratio.** Calculates how close the classes are, for each feature. Quantifies the maximum linear separability achieved between any pair of classes when projected onto the single most informative feature (or feature combination). Roughly, it indicates how easy it is to draw a straight line that separates the classes using the best single feature available.<br>Calculates for each feature the ratio of two components: a) Squared difference in the means of two classes, and b) The sum of the variances of the data in each class This maximizes the distance between classes while minimizing the spread within each class. F1 is the maximum of these feature ratios (how separable the dataset is along its single best feature axis). Lorena (and Python's *problexity*) invert this score. | **Higher Complexity.** No single feature separates (discriminates) classes well. Class distributions overlap heavily in every feature space. | ≈0 | 1 |
| **F1v** | **Directional Vector Maximum Fisher's Discriminant Ratio.** It complements F1, looking for the hyperplane, generated by a vector, that best separates the classes instead of using the separate features. Also similar to F1, score is inverted: F1v = 1 / (1 + Directional Fisher's Discriminant Ratio). | **Higher Complexity.** Dataset is so heavily mixed that a linear boundary cannot achieve good separation, even in the most helpful direction. | ≈0 | 1 |
| **F2** | **Volume of Overlapping Region.** Estimates the degree of overlap between the class distributions of each feature (dimension). This is done by calculating the normalized overlap for each feature individually (based on a feature's min and max values within an individual class) and then taking the product of the normalized values across all features. This final product represents the overall volume of the feature space where classes heavily intermingle, directly measuring the ambiguity of the raw features.<br>Note from [1]: The value is 0 if there is at least one feature in which value ranges of two classes do not overlap. | **Higher Complexity.** Greater overlap suggests less clear class boundaries, showing that the classes (e.g., 0 and 1) are thoroughly intermingled. | 0 | 1 |
| **F3** | **Maximum Individual Feature Efficiency.** Estimates the individual efficiency of each feature in separating the classes. Specifically, it quantifies the proportion of samples that fall into the overlapping (ambiguous) region for each feature, and determines the feature with the maximum value among all the features. Here Lorena et al. [2] (and *problexity*) takes the | **Higher Complexity.** Nearly all, or all, feature values of separate classes overlap; even the best single feature struggles to separate the classes. | 0 | 1 |

| Acronym | Name and Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| | complement of this measure (i.e., the minimum value) so that higher values are obtained for more complex problems. The problem can be considered simpler if there is at least one feature which shows low ambiguity between the classes. | | | |
| **F4** | **Collective Feature Efficiency.** Based on an iterative use of F3 over the dataset, each time choosing the next most efficient feature and setting aside the non-overlapped points of that feature, until there are no more points or features. The final F4 score is determined by the proportion of data points that were never unambiguously discriminated throughout this entire sequential process. | **Higher Complexity.** A large proportion of data points (e.g., dataset records) remain ambiguous and hard to classify; suggests that the features do not work effectively together to clearly separate the classes | 0 | 1 |

## Category 2. Linearity Measures

These measures try to quantify to what extent the classes are linearly separable, that is, if it is possible to separate the classes by a hyperplane. [2] The assumption is that linearly separable datasets are less complex. Higher values indicate greater complexity, suggesting that linear decision boundaries may be insufficient and that non-linear models could be required for effective classification. **Figure C-2** provides a description of the three measures included in this category.

**Figure C-2. Linearity Measures**

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **L1** | **Sum of the Error Distance by Linear Programming.** Assesses if the data are linearly separable by computing the sum of the distances of incorrectly classified examples to a linear boundary used in their classification. If the value of L1 is zero, then the problem is perfectly linearly separable and can be considered simpler than a problem for which a non-linear boundary is required. [2] | **Higher Complexity.** Many data points are far from being correctly separated by a linear boundary. This suggests the classes are significantly non-linearly separable, signifying a high complexity problem in terms of linearity. | 0 | ≈1 |
| **L2** | **Error Rate of the Linear Classifier.** The actual error rate (1 - accuracy) of a simple linear classifier (problexity uses a Linear SVM) trained on the full dataset.<br>L2 has similar issues with L1 in that it does not differentiate between problems that are barely linearly separable (i.e., with a narrow margin) from those with classes that are very far apart. [2] | **Higher Complexity.** A high error rate suggests the presence of non-linear patterns or substantial overlap (i.e., dataset cannot be effectively separated by a linear model). | 0 | 1 |

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **L3** | **Non-Linearity of a Linear Classifier.** Creates a new dataset, randomly interpolating pairs of training examples of the same class. Then, a linear classifier is trained on the original data and has its error rate measured in the new data points. This index is sensitive to how the data from a class are distributed in the border regions and on how much the convex hulls which delimit the classes overlap. [2] | **Higher Complexity.** Boundaries are non-linear or highly ambiguous, confusing the linear classifier. Classes are internally "messy" or intertwined, making them highly non-linear. | 0 | 1 |

## Category 3. Neighborhood Measures

These measures try to capture the shape of the decision boundary and characterize the class overlap by analyzing local neighborhoods of the data points. Some of them also capture the internal structure of the classes. All of them workover a distance matrix storing the distances between all pairs of points in the dataset. [2]

These measures evaluate the complexity of class boundaries by analyzing the proximity and distribution of neighboring data points. These measures determine distances between neighborhoods, overlaps, and decision boundaries. They are particularly relevant for assessing the behavior of instance-based classifiers such as k-nearest neighbors (k-NN), which rely on local structure to make predictions. **Figure C-3** provides a description of the six measures included in this category.

**Figure C-3. Neighborhood Measures**

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **N1** | **Fraction of Borderline Points.** The fraction of data points over all data points whose nearest neighbor belongs to a *different* class in a Minimum Spanning Tree (MST). The MST is examined to identify edges that connect two examples belonging to opposite classes ($y_i \neq y_j$). N1 is the fraction of data points (vertices) that connect to at least one of these opposite-class edges. These points are considered borderline because they are the closest neighbors that belong to different classes, indicating a tight boundary, an overlap, or noisy data. | **Higher Complexity.** Indicates high class overlap and diffuse, interwoven boundaries. The decision boundary would need to be highly complex or convoluted to separate these classes effectively. Could indicate a case where erroneous class labels are introduced during data preparation. [2] | 0 | 1 |
| **N2** | **Ratio of Intra/Extra Class Nearest Neighbor (NN) Distance.** The ratio of the average distance between a point and its nearest neighbor of the *same* class (intra) to the average distance to its nearest neighbor of a *different* class (extra). Measures if points from the same class are closer to each other (or not) than they are to points from a different class. | **Higher Complexity.** Indicates classes are locally mixed and heavily overlap, leading to high ambiguity and noisy boundaries | 0 | ≈1 |

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **N3** | **Error Rate of NN Classifier.** The error rate (1 - accuracy) of a 1-NN classifier, often calculated using leave-one-out cross-validation method.* Note from [1]: The proximity of points in opposite classes obviously affects the error rate of a NN classifier.<br>* Each data point is classified using all the other points as the training set, with its own record left out, so the error rate reflects how well the classifier generalizes at the most granular level. | **Higher Complexity.** Indicates that the class boundaries are highly non-linear and difficult for a simple distance-based approach, resulting in a higher error rate. | 0 | 1 |
| **N4** | **Non-linearity of NN Classifier.** Measures the error rate of the 1-NN classifier on a set of synthetic points interpolated between same-class pairs. Synthetic points are calculated at the midpoint between every pair of points that belong to the same class. These points should ideally also be classified the same as the two end points. Like L3 but using 1-NN instead of a linear model. | **Higher Complexity.** Indicates that the local class boundaries are excessively jagged or ambiguous. Many samples (records) intrude into the “wrong” class’s territory. | 0 | 1 |
| **T1** | **Fraction of Hyperspheres Covering Data.** Measures how fragmented and tangled the class regions are. It estimates the difficulty of separating the classes by calculating how many distinct, single-class "islands" must be individually identified to cover all the data points. It uses a geometric process: a) Hypersphere growth - expanding the radius of each sphere around every data point until it reaches a datapoint of another class, b) Sphere elimination - eliminating small spheres contained in larger spheres, and c) T1 calculation: ratio of the number of remaining, non-overlapping spheres to total number of data points. | **Higher Complexity.** Many small spheres remain. Data points of one class are deeply surrounded by the other class. The decision boundary needed to separate these small, intermixed regions is complex, nonlinear, and difficult for algorithms to model accurately. | 0 | 1 |

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **LSC** | **Local Set Average Cardinality.** Measures the average local density of same-class neighbors around each data point. It measures how isolated an instance is from its own class compared to the proximity of the opposite class. The measure uses a distance-based mechanism for every data point (x).<br>the average of the set of points whose distance is smaller than the distance of the other class. (*n* = number of records or instances) Lorena et al. inverts the LSC result, so the case of low actual LSC (data points are scattered close to the boundary) gets a high score.<br>$$LSC = \frac{\sum \lvert L_{x_i} \rvert}{n^2}$$<br>Where $\lvert L_{xi} \rvert$ is the cardinality of local set for instance $x_i$, and *n* is the total number of instances in the dataset. | **Higher Complexity.** Indicates local neighborhoods are highly mixed and overlapping, with many points from other classes. Data points are extremely close to the decision boundary. | 0 | $1 - \frac{1}{n}$ |

## Category 4. Network Measures

This category uses graph theory to characterize the topological structure of the dataset by modeling data points as nodes and local neighborhood relationships as connections. These measures quantify properties like local connectivity, cliquishness, and sparsity. Analyzing this graph structure reveals how fragmented or cohesive the underlying class boundaries are, offering insight into how tightly grouped or dispersed the data are.

Crucially, Lorena et al. [2] invert the scores for all three measures from their standard definitions. This normalization ensures that a higher score consistently shows higher complexity, specifically due to a lack of local structure, greater dispersion, or increased fragmentation within the dataset's neighborhoods. **Figure C-4**, on the following page, provides a description of the three measures included in this category.

**Figure C-4. Network Measures**

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **Density** | **Average Density of the Network.** Quantifies how fully connected the dataset's neighborhood graph is. It is derived by comparing the actual number of existing links (edges) between data points (nodes) within a defined local neighborhood to the maximum possible number of links that could exist. It measures how tightly clustered and connected points of the same class are in the local neighborhood. The Lorena et al. Density measure uses an inverted and normalized calculation to ensure that lower density in the graph structure correctly maps to higher complexity. | **Higher Complexity.** Fewer edges observed for datasets of low density (examples are far apart in the input space) and/or for which examples of opposite classes are near each other, imply higher complexity. | 0 | 1 |
| **ClsCoef** | **Average Clustering Coefficient.** Quantifies the cliquishness (or tendency to form closed triangles) within the local neighborhoods of the data graph. It assesses the degree of local connectivity, which is then inverted to quantify complexity.<br>It is derived as the mean of the ratio between the actual number of closed triangles (cliques) centered on a point and the maximum possible number of triangles that could exist with its neighbors. This calculation assesses the local cohesiveness or clumpiness of the dataset's graph structure. | **Higher Complexity.** Data points in a neighborhood are not tightly connected to one another. There are few "closed triangles" or cliques, indicating higher complexity. | 0 | 1 |
| **Hubs** | **Mean Hub Score.** This is a measure of the influence of each node of the graph. [2] It assesses the concentration of influence within the dataset's graph structure by quantifying how often a few points ("hubs") are chosen as nearest neighbors by others. A Hub is a data point that is a nearest neighbor to a significantly larger number of other points than the average. This measure captures the structural homogeneity of the dataset. With the Lorena et al. inversion, a low score for this concentration suggests a stable, centralized structure (low complexity), while a high score suggests a dispersed, fragile structure (high complexity). | **Higher Score Complexity.** The structure is less centralized, more dispersed, and offers less robust local guidance to model. | 0 | 1 |

## Category 5. Dimensionality Measures

This group captures the potential for data sparsity arising from high dimensionality compared to the number of samples. Higher values indicate greater complexity, suggesting that models may meet challenges in localized or underpopulated regions of the feature space. **Figure C-5**, on the following page, provides a description of the three measures included in this category.

**Figure C-5. Dimensionality Measures**

| Acronym | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **T2** | **Average Number of Features per Point.** Defined as the ratio of the number of features to the number of data points in the dataset. Reflects data sparsity: if there are many predictive attributes and few data points, they will probably be sparsely distributed in the input space. (*m* = number of features) | **Higher Complexity.** More features per point will increase sparsity, showing higher complexity. | ≈0 | *m* |
| **T3** | **Average Number of PCA Dimensions per Point.** Measures the dataset's intrinsic dimensionality relative to its sample size. It is calculated as the ratio of the number of essential Principal Component Analysis (PCA)[a] dimensions (PCA-D) needed to explain 95% of the data's variance to the total number of records in the dataset. This measure represents the true, necessary dimensionality of the data after removing redundancy. (*m* = number of features) | **Higher Complexity.** Even when reduced by PCA, the ratio of required dimensions to records remains high, indicating sparsity. | ≈0 | *m* |
| **T4** | **Ratio of the PCA Dimension to the Original Dimension.** The ratio of the intrinsic dimensionality (from PCA) to the original number of features. This quantifies how many of the original features are genuinely needed to explain the data's variance (the intrinsic dimensionality) versus the total number of features available. | **Higher Complexity.** If nearly all the original features are needed to describe the data's variability, complexity is greater. These relationships between the input variables are complex, redundant, or highly scattered. | 0 | 1 |

[a] PCA reduces dimensionality by projecting data onto orthogonal directions that capture maximal variance in a dataset, and offers some insight into a dataset's intrinsic complexity

## Category 6. Class Imbalance Measures

These measures quantify disparities in the number of samples in each class, which can lead to model bias toward the majority class. The two measures help characterize the extent to which class imbalance may affect classification performance. According to Lorena et al., when the differences are severe, most of the ML classification techniques tend to favor the majority class and present generalization problems. [2]

Lorena et al. [2] use generalized and normalized formulas for C1 and C2, rather than their raw statistical definitions. This method ensures both measures are correctly oriented, meaning that a higher score consistently indicates a higher level of imbalance complexity for the algorithm. Thus, both measures provide a standardized, comparable assessment of the penalty imposed on classifiers by skewed class distributions. **Figure C-6**, on the following page, provides a description of the two measures included in this category.

**Figure C-6. Class Imbalance Measures**

| Measure | Definition | Interpretation | Min | Max |
|---|---|---|---|---|
| **C1** | **Entropy of Class Proportions.** Derived from Shannon Entropy, then normalized and inverted to quantify the Imbalance difficulty of the class distribution. When classes are perfectly balanced (i.e., 50% v. 50%), C1 score is lowest. When one class dominates completely (e.g., 99% v. 1%), C1 score is highest, meaning there is no uncertainty about the class label. A model can simply guess the majority class. | **Higher Complexity.** The less balanced the classes, the higher the complexity. | 0 | 1 |
| **C2** | **Imbalance Ratio.** Measures the degree of disparity in the number of samples per class, using a generalized formula designed to work consistently for both binary and multiclass classification problems. The value ranges from 0 for perfectly balanced datasets to nearly 1 for extreme imbalance, imposing a significant difficulty penalty. The formula simplifies for a binary classification task to: $$C2 = 1 - \frac{2n_1 n_2}{n_1^2 n_2^2}$$ Here, n1 and n2 represent the number of records belonging to the two respective classes. | **Higher Complexity.** Indicates a highly imbalanced dataset. | 0 | 1 |

# Appendix B – Dataset Characteristics and Sources

**Exhibit C-1**, on the following page, catalogs the 182 datasets used in the benchmarking study. For each entry, the exhibit lists the name, record count, feature count, and source. Also shown are three analytical indicators: the overall complexity score, mean accuracy (across all 12 models), and the dominant complexity category for each dataset. These indicators show how a dataset's structural properties contribute to classification difficulty, referencing the complexity analysis presented in the main report.

The Dominant Category column names which of the six complexity categories (as defined by Lorena et al. [2]) best characterizes each dataset. This category is found by calculating the mean standardized score for each group of complexity measures and then selecting the group with the maximum mean, which is reported alongside the category name (in parentheses). For example, a dataset is classified as "Feature-based" if the average of its five feature-based measures (F1, F1v, F2, F3, F4) was the highest category mean.

Selected statistics from the dataset catalog include:

- Most records (1,138,289): ACSPublicCoverage
- Least records (32): parity5
- Most features (2,569): ember2024_50000
- Least features (2): prnn_synth, make_blobs_dataset, make_circles_dataset, and make_moons_dataset
- Highest complexity score (0.683): parity5
- Lowest complexity score (0.182): kddcup99
- Highest mean accuracy (0.999): creditcard, online_payments_fraud_detection, kddcup99
- Lowest mean accuracy (0.333): analcatdata_aids
- Highest maximum category score (1.000): parity 5, Feature-based category
- Lowest maximum category score (0.223): PhiUSIILPhishingURL-Website, Neighborhood
- Number of occurrences in Dominant Category:

| | |
|---|---|
| Feature-based | 20 |
| Linearity | 26 |
| Neighborhood | 53 |
| Network | 22 |
| Dimensionality | 44 |
| Class Imbalance | 17 |
| **Total** | **182** |

### Exhibit C-1 Dataset Catalog and Characteristics

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| ACSIncome | 1,000,000 | 11 | OpenML | 0.532 | 0.733 | Feature-based (0.895) |
| ACSPublicCoverage | 1,138,289 | 18 | OpenML | 0.299 | 0.950 | Class Imbalance (0.716) |
| adult_income | 45,222 | 15 | UCI | 0.424 | 0.851 | Feature-based (0.642) |
| ailerons | 13,750 | 41 | OpenML | 0.333 | 0.834 | Feature-based (0.510) |
| air_quality_and_pollution_assessment | 5,000 | 10 | Kaggle | 0.295 | 0.952 | Class Imbalance (0.656) |
| airlines_delay | 539,383 | 7 | OpenML | 0.546 | 0.640 | Feature-based (0.964) |
| albert | 58,252 | 32 | OpenML | 0.517 | 0.636 | Feature-based (0.727) |
| Amazon_access | 32,769 | 10 | OpenML | 0.480 | 0.943 | Feature-based (0.806) |
| analcatdata_aids | 50 | 5 | OpenML | 0.504 | 0.333 | Neighborhood (0.609) |
| analcatdata_asbestos | 83 | 4 | OpenML | 0.406 | 0.725 | Feature-based (0.679) |
| analcatdata_bankruptcy | 50 | 7 | OpenML | 0.304 | 0.492 | Neighborhood (0.492) |
| analcatdata_boxing1 | 120 | 4 | OpenML | 0.461 | 0.660 | Feature-based (0.757) |
| analcatdata_boxing2 | 132 | 4 | OpenML | 0.499 | 0.728 | Feature-based (0.892) |
| analcatdata_creditscore | 100 | 7 | OpenML | 0.304 | 0.825 | Dimensionality (0.373) |
| analcatdata_cyyoung8092 | 97 | 11 | OpenML | 0.322 | 0.708 | Neighborhood (0.397) |
| analcatdata_cyyoung9302 | 92 | 11 | OpenML | 0.316 | 0.653 | Class Imbalance (0.389) |
| analcatdata_fraud | 42 | 12 | OpenML | 0.472 | 0.583 | Feature-based (0.560) |
| analcatdata_japansolvent | 52 | 10 | OpenML | 0.299 | 0.674 | Neighborhood (0.518) |
| analcatdata_lawsuit | 264 | 5 | OpenML | 0.285 | 0.898 | Class Imbalance (0.737) |
| appendicitis | 106 | 8 | OpenML | 0.329 | 0.769 | Class Imbalance (0.408) |
| australian | 690 | 15 | OpenML | 0.403 | 0.844 | Feature-based (0.542) |
| backache | 180 | 32 | OpenML | 0.449 | 0.801 | Class Imbalance (0.553) |
| BAF_base | 1,000,000 | 29 | OpenML | 0.470 | 0.989 | Class Imbalance (0.946) |
| BAF_variant_III | 1,000,000 | 31 | OpenML | 0.466 | 0.992 | Class Imbalance (0.946) |
| BankCustomerChurnPrediction | 10,000 | 12 | Kaggle | 0.501 | 0.850 | Feature-based (0.761) |
| bank_marketing_data | 45,211 | 13 | UCI | 0.418 | 0.896 | Class Imbalance (0.610) |
| banknote_authentication | 1,372 | 5 | UCI | 0.242 | 0.828 | Feature-based (0.304) |
| BaysianNetwGen_mushroom | 1,000,000 | 22 | OpenML | 0.362 | 0.910 | Feature-based (0.739) |
| BEED | 8,000 | 17 | UCI | 0.317 | 0.888 | Feature-based (0.602) |
| biomed | 209 | 9 | OpenML | 0.325 | 0.829 | Neighborhood (0.329) |
| bioresponse | 3,434 | 420 | OpenML | 0.407 | 0.703 | Feature-based (0.697) |
| blood_transfusion_classification | 748 | 5 | UCI | 0.446 | 0.687 | Feature-based (0.731) |
| BNG_breast-w | 39,366 | 10 | OpenML | 0.285 | 0.983 | Feature-based (0.530) |
| BNG_cmc | 55,296 | 10 | OpenML | 0.499 | 0.791 | Feature-based (0.841) |
| BNG_colic | 1,000,000 | 22 | OpenML | 0.447 | 0.889 | Feature-based (0.636) |

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| BNG_cylinder_bands | 1,000,000 | 38 | OpenML | 0.485 | 0.829 | Feature-based (0.748) |
| BNG_heart-satlog | 1,000,000 | 13 | OpenML | 0.406 | 0.899 | Feature-based (0.657) |
| BNG_hepatitis | 1,000,000 | 19 | OpenML | 0.419 | 0.937 | Feature-based (0.629) |
| BNG_kr_vs_kp | 1,000,000 | 36 | OpenML | 0.463 | 0.918 | Feature-based (0.818) |
| BNG_labor | 1,000,000 | 16 | OpenML | 0.389 | 0.948 | Feature-based (0.604) |
| BNG_Paribus_Cardif | 17,756 | 131 | OpenML | 0.496 | 0.708 | Feature-based (0.777) |
| breast_cancer_dataset | 569 | 31 | Kaggle | 0.209 | 0.885 | Neighborhood (0.273) |
| breast_cancer_prediction | 569 | 31 | UCI | 0.208 | 0.873 | Neighborhood (0.271) |
| california_environment_conditions | 128,009 | 18 | OpenML | 0.396 | 0.873 | Class Imbalance (0.832) |
| california_housing_classification | 20,640 | 9 | OpenML | 0.356 | 0.841 | Feature-based (0.602) |
| cardiovascular_disease_classification | 70,000 | 13 | OpenML | 0.504 | 0.720 | Feature-based (0.838) |
| churn | 5,000 | 21 | OpenML | 0.478 | 0.895 | Feature-based (0.661) |
| clean1 | 476 | 169 | OpenML | 0.372 | 0.721 | Feature-based (0.461) |
| cleve | 303 | 14 | OpenML | 0.441 | 0.769 | Feature-based (0.605) |
| Click | 1,000,000 | 11 | OpenML | 0.413 | 0.745 | Feature-based (0.616) |
| cmc | 1,473 | 10 | OpenML | 0.501 | 0.636 | Feature-based (0.843) |
| codrna | 488,565 | 9 | OpenML | 0.366 | 0.964 | Feature-based (0.688) |
| college_scorecard | 122,279 | 117 | OpenML | 0.420 | 0.937 | Feature-based (0.650) |
| company_bankruptcy_prediction | 6,819 | 96 | Kaggle | 0.397 | 0.962 | Class Imbalance (0.864) |
| compass | 16,644 | 18 | OpenML | 0.455 | 0.690 | Feature-based (0.704) |
| comprehensive_diabetes_clinical | 100,000 | 16 | Kaggle | 0.345 | 0.968 | Class Imbalance (0.698) |
| contraceptive_method_choice | 1,473 | 10 | UCI | 0.502 | 0.640 | Feature-based (0.843) |
| corral | 160 | 7 | OpenML | 0.402 | 0.758 | Feature-based (0.747) |
| covertype | 566,602 | 11 | OpenML | 0.476 | 0.851 | Feature-based (0.773) |
| credit | 16,714 | 11 | OpenML | 0.421 | 0.721 | Feature-based (0.723) |
| credit_approval | 690 | 16 | UCI | 0.369 | 0.764 | Feature-based (0.468) |
| credit_g | 1,000 | 21 | OpenML | 0.527 | 0.711 | Feature-based (0.798) |
| creditcard | 284,807 | 30 | OpenML | 0.226 | 0.999 | Class Imbalance (0.990) |
| Customer_Churn_Class | 174,959 | 24 | OpenML | 0.441 | 0.853 | Feature-based (0.657) |
| default_of_credit_card_clients | 30,000 | 24 | UCI | 0.433 | 0.813 | Feature-based (0.664) |
| diabetes_health_indicators | 253,680 | 22 | UCI | 0.505 | 0.867 | Feature-based (0.828) |
| diabetes130US | 71,090 | 8 | OpenML | 0.509 | 0.608 | Feature-based (0.790) |
| diabetic_retinopathy_debrecen | 1,151 | 20 | UCI | 0.420 | 0.597 | Feature-based (0.631) |
| digits | 1,797 | 65 | Other | 0.341 | 0.969 | Class Imbalance (0.657) |
| e_commerce_shipping_data | 10,999 | 12 | Kaggle | 0.448 | 0.678 | Feature-based (0.535) |
| egg_eye_state | 14,980 | 15 | OpenML | 0.374 | 0.589 | Feature-based (0.731) |

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| electricity | 38,474 | 8 | OpenML | 0.411 | 0.771 | Feature-based (0.663) |
| elevators | 16,599 | 19 | OpenML | 0.383 | 0.879 | Feature-based (0.558) |
| ember2024_50000 | 50,000 | 2,569 | Other | 0.387 | 0.807 | Feature-based (0.645) |
| fico_heloc | 9,871 | 24 | OpenML | 0.449 | 0.727 | Feature-based (0.616) |
| fitness_class_2212 | 1,467 | 7 | Kaggle | 0.424 | 0.735 | Feature-based (0.612) |
| gallstone | 319 | 39 | UCI | 0.424 | 0.647 | Neighborhood (0.526) |
| gina_agnostic | 3,468 | 971 | OpenML | 0.435 | 0.847 | Feature-based (0.574) |
| GiveMeSomeCredit | 149,391 | 10 | OpenML | 0.350 | 0.934 | Class Imbalance (0.750) |
| haberman | 306 | 4 | OpenML | 0.506 | 0.679 | Feature-based (0.834) |
| heart_disease_dataset_comprehensive | 1,190 | 12 | OpenML | 0.379 | 0.751 | Feature-based (0.583) |
| heart-statlog | 270 | 14 | OpenML | 0.419 | 0.773 | Feature-based (0.571) |
| heloc | 10,000 | 23 | OpenML | 0.447 | 0.719 | Feature-based (0.658) |
| hepmass_all_train | 1,000,000 | 28 | UCI | 0.425 | 0.869 | Feature-based (0.522) |
| higgs | 19,999 | 29 | OpenML | 0.516 | 0.660 | Feature-based (0.737) |
| higgs_boson_2014 | 218,577 | 34 | UCI | 0.314 | 0.958 | Neighborhood (0.464) |
| hill_valley | 1,212 | 101 | OpenML | 0.410 | 0.492 | Feature-based (0.722) |
| house_16h | 13,488 | 17 | OpenML | 0.384 | 0.841 | Feature-based (0.620) |
| HTRU2 | 17,898 | 9 | UCI | 0.268 | 0.979 | Class Imbalance (0.679) |
| in_vehicle_coupon_recommendation | 12,684 | 26 | UCI | 0.529 | 0.678 | Feature-based (0.877) |
| insurance | 23,548 | 11 | OpenML | 0.541 | 0.747 | Feature-based (0.980) |
| Invistico_Airline | 129,880 | 22 | Kaggle | 0.394 | 0.940 | Feature-based (0.558) |
| ionosphere | 351 | 35 | OpenML | 0.311 | 0.803 | Neighborhood (0.344) |
| iranian_churn | 3,150 | 14 | UCI | 0.329 | 0.842 | Class Imbalance (0.507) |
| iris | 150 | 5 | OpenML | 0.282 | 0.825 | Feature-based (0.323) |
| irish | 500 | 6 | OpenML | 0.297 | 0.841 | Feature-based (0.478) |
| is_this_a_good_customer | 1,723 | 14 | OpenML | 0.487 | 0.884 | Feature-based (0.708) |
| jannis | 57,580 | 55 | OpenML | 0.478 | 0.751 | Feature-based (0.639) |
| jasmine | 2,984 | 145 | OpenML | 0.460 | 0.692 | Feature-based (0.691) |
| jm1 | 10,885 | 22 | OpenML | 0.397 | 0.807 | Feature-based (0.695) |
| kc1 | 2,109 | 22 | OpenML | 0.348 | 0.842 | Feature-based (0.603) |
| KDD | 5,032 | 46 | OpenML | 0.467 | 0.692 | Feature-based (0.666) |
| kdd_ipums_la_97_small | 5,188 | 21 | OpenML | 0.290 | 0.804 | Feature-based (0.368) |
| kddcup99 | 25,000 | 42 | OpenML | 0.182 | 0.999 | Class Imbalance (0.975) |
| liver-disorders | 345 | 7 | OpenML | 0.467 | 0.628 | Feature-based (0.700) |
| LT-Vehicle_Loan_Default_Pred | 225,493 | 38 | OpenML | 0.480 | 0.780 | Feature-based (0.765) |
| lupus | 87 | 4 | OpenML | 0.400 | 0.718 | Feature-based (0.543) |
| madeline | 3,140 | 260 | OpenML | 0.545 | 0.551 | Feature-based (0.737) |

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| MagicTelescope | 19,020 | 12 | OpenML | 0.213 | 0.988 | Neighborhood (0.255) |
| make_blobs_dataset | 20,000 | 3 | Other | 0.357 | 0.855 | Feature-based (0.591) |
| make_circles_dataset | 20,000 | 3 | Other | 0.414 | 0.949 | Feature-based (0.766) |
| make_classification | 20,000 | 31 | Other | 0.484 | 0.910 | Feature-based (0.609) |
| make_moons_dataset | 20,000 | 3 | Other | 0.291 | 0.922 | Feature-based (0.405) |
| mammography | 11,183 | 7 | OpenML | 0.361 | 0.984 | Class Imbalance (0.897) |
| marketing_campaign | 2,240 | 29 | Kaggle | 0.446 | 0.803 | Feature-based (0.595) |
| medical_appointment_no_shows_convert_dates | 110,527 | 20 | OpenML | 0.463 | 0.799 | Feature-based (0.743) |
| Medical-Appointment | 61,214 | 19 | OpenML | 0.549 | 0.796 | Feature-based (0.872) |
| miniboone | 72,998 | 51 | OpenML | 0.315 | 0.857 | Feature-based (0.661) |
| multiple_sclerosis_disease_cols_removed | 273 | 17 | Kaggle | 0.433 | 0.720 | Feature-based (0.561) |
| mux6 | 128 | 7 | OpenML | 0.482 | 0.731 | Feature-based (0.862) |
| nasa_nearest_object_valued | 338,199 | 6 | Kaggle | 0.332 | 0.883 | Class Imbalance (0.581) |
| naticusdroid_android_permissions | 29,332 | 87 | UCI | 0.340 | 0.924 | Feature-based (0.546) |
| nhanes_age_predictions_subset | 2,278 | 8 | UCI | 0.430 | 0.810 | Feature-based (0.681) |
| numerai28.6 | 96,320 | 22 | OpenML | 0.590 | 0.517 | Feature-based (0.997) |
| occupancy_detection_combined_date | 20,560 | 10 | UCI | 0.248 | 0.976 | Class Imbalance (0.335) |
| online_payments_fraud_detection | 800,000 | 7 | OpenML | 0.284 | 0.999 | Class Imbalance (0.992) |
| online_shoppers | 12,330 | 18 | OpenML | 0.404 | 0.878 | Feature-based (0.581) |
| ozone-level-8hr | 2,534 | 73 | OpenML | 0.378 | 0.924 | Class Imbalance (0.763) |
| page-blocks | 5,473 | 11 | OpenML | 0.305 | 0.951 | Class Imbalance (0.650) |
| parity5 | 32 | 6 | OpenML | 0.683 | 0.369 | Feature-based (1.000) |
| pc1 | 1,109 | 22 | OpenML | 0.314 | 0.898 | Class Imbalance (0.744) |
| pc3 | 1,563 | 38 | OpenML | 0.330 | 0.814 | Class Imbalance (0.650) |
| pc4 | 1,458 | 38 | OpenML | 0.326 | 0.853 | Class Imbalance (0.596) |
| philippine | 5,832 | 309 | OpenML | 0.491 | 0.656 | Feature-based (0.629) |
| PhiUSIILPhishingURL-Website | 25,000 | 52 | UCI | 0.183 | 0.964 | Neighborhood (0.223) |
| phoneme | 5,404 | 6 | OpenML | 0.419 | 0.808 | Feature-based (0.641) |
| pima_indians_diabetes | 768 | 9 | UCI | 0.452 | 0.676 | Feature-based (0.642) |
| pol | 10,082 | 27 | OpenML | 0.308 | 0.904 | Feature-based (0.476) |
| pollen | 3,848 | 6 | OpenML | 0.548 | 0.493 | Feature-based (0.902) |
| porto-seguro | 124,931 | 57 | OpenML | 0.530 | 0.955 | Class Imbalance (0.819) |
| postoperative-patient-data | 88 | 9 | OpenML | 0.547 | 0.787 | Feature-based (0.805) |
| predict_students_dropout_and_academic_success | 4,424 | 37 | UCI | 0.403 | 0.779 | Feature-based (0.547) |
| prnn_crabs | 200 | 8 | OpenML | 0.296 | 0.806 | Feature-based (0.526) |
| prnn_synth | 250 | 3 | OpenML | 0.319 | 0.783 | Feature-based (0.443) |

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| pumpkin_seeds | 2,500 | 13 | OpenML | 0.301 | 0.840 | Feature-based (0.446) |
| qsar | 1,055 | 41 | OpenML | 0.361 | 0.754 | Feature-based (0.523) |
| qsar-biodeg | 1,054 | 42 | OpenML | 0.359 | 0.820 | Feature-based (0.521) |
| Raisin | 900 | 8 | UCI | 0.288 | 0.737 | Feature-based (0.443) |
| rice_cammeo_and_osmancik | 3,810 | 8 | UCI | 0.255 | 0.919 | Feature-based (0.330) |
| ringnorm | 7,400 | 21 | OpenML | 0.457 | 0.904 | Feature-based (0.575) |
| rl | 4,970 | 13 | OpenML | 0.499 | 0.603 | Feature-based (0.798) |
| road_safety | 111,762 | 33 | OpenML | 0.462 | 0.743 | Feature-based (0.661) |
| sa-heart | 462 | 10 | OpenML | 0.481 | 0.674 | Feature-based (0.679) |
| sandtander_customer_satisfaction | 25,000 | 201 | OpenML | 0.562 | 0.906 | Feature-based (0.744) |
| satellite | 5,100 | 37 | OpenML | 0.296 | 0.988 | Class Imbalance (0.930) |
| segment | 2,310 | 20 | OpenML | 0.258 | 0.935 | Class Imbalance (0.542) |
| sepsis_survival_minimal_clinical_records | 110,204 | 4 | UCI | 0.507 | 0.927 | Feature-based (0.930) |
| sf_police_incidents | 538,638 | 6 | OpenML | 0.562 | 0.581 | Feature-based (0.992) |
| shipping | 10,999 | 10 | OpenML | 0.449 | 0.658 | Feature-based (0.574) |
| shrutime | 10,000 | 11 | OpenML | 0.492 | 0.851 | Feature-based (0.761) |
| sick | 3,103 | 23 | OpenML | 0.369 | 0.920 | Class Imbalance (0.745) |
| smoking_drinking_dataset_Ver01 | 100,000 | 24 | Kaggle | 0.432 | 0.731 | Feature-based (0.629) |
| sonar | 208 | 61 | OpenML | 0.379 | 0.742 | Neighborhood (0.431) |
| spambase | 4,601 | 58 | OpenML | 0.308 | 0.865 | Feature-based (0.539) |
| spect | 267 | 23 | OpenML | 0.435 | 0.773 | Feature-based (0.540) |
| spectf | 349 | 45 | OpenML | 0.365 | 0.757 | Neighborhood (0.386) |
| stroke_prediction_dataset | 4,909 | 11 | Kaggle | 0.429 | 0.962 | Class Imbalance (0.829) |
| student_depression_dataset | 27,901 | 17 | OpenML | 0.403 | 0.831 | Feature-based (0.547) |
| susy | 1,000,000 | 18 | UCI | 0.428 | 0.800 | Feature-based (0.599) |
| svmguide3 | 1,243 | 23 | OpenML | 0.405 | 0.723 | Feature-based (0.593) |
| sylvine | 5,124 | 21 | OpenML | 0.396 | 0.870 | Neighborhood (0.475) |
| taiwanese_bankruptcy_prediction | 6,819 | 96 | UCI | 0.397 | 0.962 | Class Imbalance (0.864) |
| TCGA_GBM_LGG_Mutations_all | 862 | 26 | UCI | 0.203 | 0.875 | Dimensionality (0.298) |
| telecom_customer_churn | 7,043 | 20 | Kaggle | 0.478 | 0.779 | Feature-based (0.809) |
| twonorm | 7,400 | 21 | OpenML | 0.397 | 0.969 | Feature-based (0.529) |
| vehicleNorm | 25,000 | 101 | OpenML | 0.409 | 0.829 | Feature-based (0.608) |
| visualizing_soil | 8,641 | 5 | OpenML | 0.227 | 0.969 | Dimensionality (0.333) |
| vulnonevul | 5,692 | 17 | OpenML | 0.329 | 0.986 | Class Imbalance (0.953) |
| waterQuality1 | 7,996 | 21 | Kaggle | 0.502 | 0.927 | Feature-based (0.754) |
| waveform-5000 | 5,000 | 41 | OpenML | 0.449 | 0.849 | Feature-based (0.560) |

| Dataset | Records | Features | Source | Complex-ity Score | Mean Accuracy | Dominant Category |
|---|---|---|---|---|---|---|
| wilt_seed | 2,000 | 6 | OpenML | 0.312 | 0.898 | Class Imbalance (0.792) |
| wine_quality | 6,497 | 12 | UCI | 0.482 | 0.560 | Feature-based (0.766) |
| yeast_ml8 | 2,417 | 117 | OpenML | 0.468 | 0.978 | Class Imbalance (0.932) |

# Appendix C - Computational Infrastructure

Model training and evaluation were conducted in Google Colab using an NVIDIA A100-SXM4-80GB GPU. Results analysis was performed locally in Visual Studio Code using Jupyter notebooks on a Windows PC. The evaluation pipeline ran on Python 3.13.3 with scikit-learn 1.7.0. Standard data science libraries were used throughout, including pandas for data manipulation and file I/O.

All 12 models used a single train/validation/test split per dataset. For the eight *pytorch-tabular* models, data was split 80% training and 20% testing. For the four *pytorch-widedeep* models, the training portion was further divided, reserving 20% of the training data for validation, resulting in a 64% training, 16% validation, and 20% testing split. All splits used random_state=42 for reproducibility.

Training for all twelve deep learning models was capped at 40 epochs. Early stopping was applied in both frameworks, monitoring validation loss with a patience of 5 epochs and a minimum improvement threshold of 0.001. Training stopped when validation loss did not improve for five consecutive epochs, preventing overfitting and avoiding unnecessary computation. **Figure C.1** summarizes the number of epochs used across the 2,172 model–dataset runs.

**Figure C.1 Epochs Used**

| Range | Count |
|---|---|
| 0-5 | 10 |
| 6-10 | 252 |
| 11-15 | 184 |
| 16-20 | 164 |
| 21-25 | 131 |
| 26-30 | 96 |
| 31-35 | 72 |
| 36-39 | 60 |
| 40 | 1,203 |
| **Total** | **2,172** |

This distribution shows that early stopping triggered often for many model–dataset pairs, but a large portion (over half) reached the full 40-epoch cap. This shows that while early stopping reduced computation for many runs, a large number of datasets required the full training budget to converge.

Model throughput was measured as predictions per second on the A100 GPU under consistent conditions across all models and datasets.

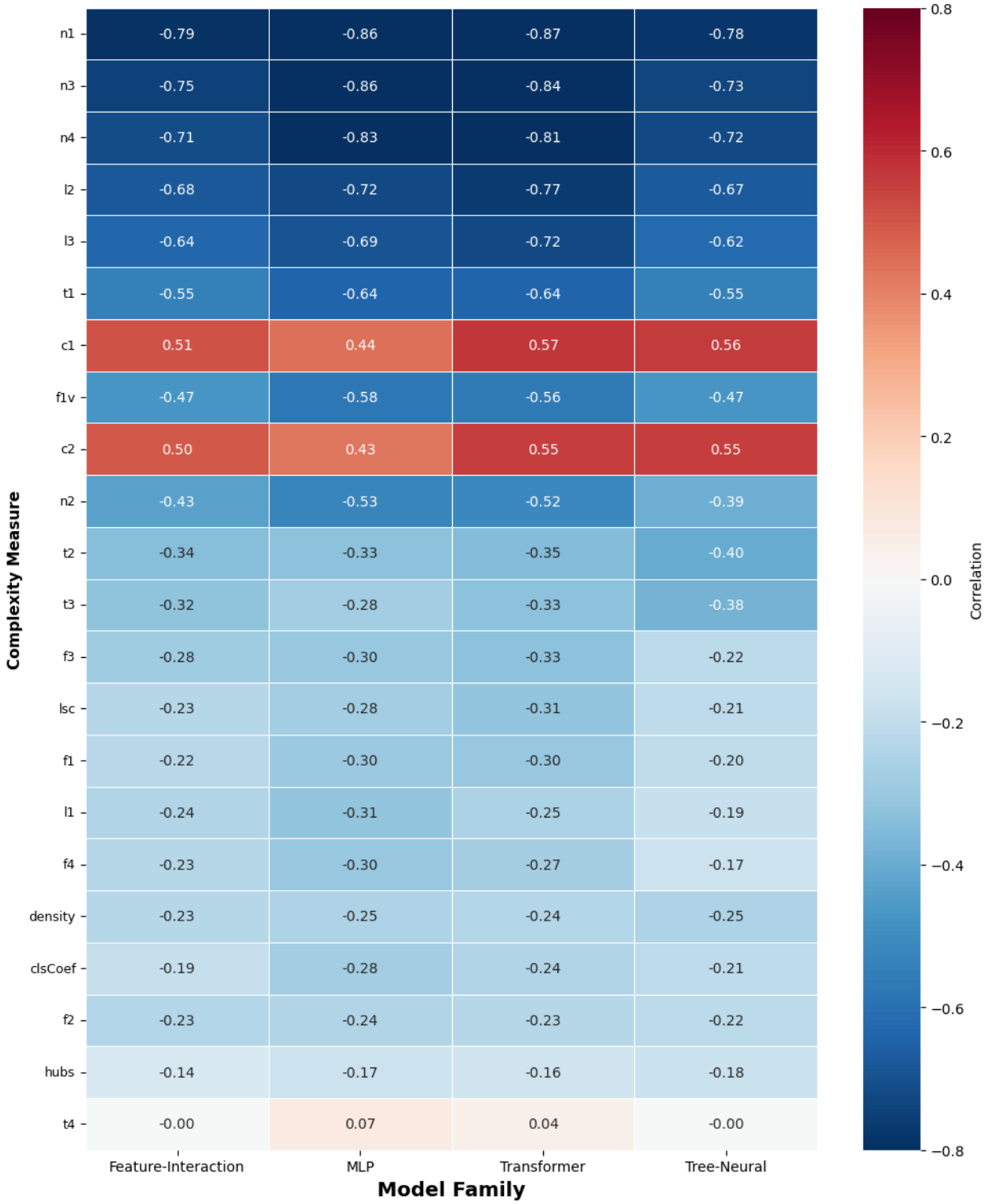
# Appendix D - Pearson Correlations Between Complexity Measures and Accuracy by Model Family

The heatmap displayed in **Exhibit D.1**, on the following page, presents Pearson correlation coefficients between each of the 22 complexity measures and model accuracy, calculated separately for each of the four model families. Each column represents one model family, showing how strongly that family's performance correlates with each complexity measure.

Some complexity measures affect different model families more strongly than others. Measures with five largest differential effects (range of correlations across families) are the following:

| | |
|---|---|
| N2 | Range: 0.136 (Min: -0.529, Max: -0.393) |
| N3 | Range: 0.130 (Min: -0.864, Max: -0.734) |
| F4 | Range: 0.130 (Min: -0.302, Max: -0.172) |
| C1 | Range: 0.129 (Min: +0.440, Max: +0.569) |
| C2 | Range: 0.128 (Min: +0.427, Max: +0.555). |

**Exhibit D.1 Complexity-Accuracy Correlations by Model Family**



# References


[1] T. K. Ho and M. Basu, "Complexity Measures of Supervised Classification Problems," *IEEE Transactions on Pattern Analysis and Machine Intelligence,* vol. 24, no. 3, pp. 289-300, 2002.

[2] A. C. Lorena, L. P. Garcia, J. Lehmann, M. C. Souto and T. K. Ho, "How complex is your classification problem? A survey on measuring classification complexity," *ACM Computing Surveys CSUR,* vol. 52, no. 5, pp. 1-34, 2018.

[3] J. D. Pascual-Triana, D. Charte, M. A. Arroyo and A. Fernandez, "Revisiting Data Complexity Metrics Based on Morphology for Overalap and Imbalance: Snapshot, New Overlap Number of Balls Metrics and Singular Problems Prospect," *Knowl Inf Syst,* pp. 1-29, 2021.

[4] S. Uddin and H. Lu, "Confirming the statistically significant superiority of tree-based machine learning algorithms over their counterparts for tabular data," *PLoS ONE,* vol. 19, no. 4 e0301541, 2024.

[5] R. L. Theriault and R. E. Ellisa, "Data Complexity Measures Can Predict Classification Accuracy," pp. 1-34, 2025.

[6] Y. Gorishniy, I. Rubachev, V. Khrulkov and A. Babenko, "Revisiting Deep Learning Models for Tabular Data," *arXiv:2106.11959 [cs.LG],* June 2021.

[7] D. C. McElfresh, S. Khandagale, J. Valverde, V. Prasad C., G. Ramakrishnan, M. Goldblum and C. White, "When Do Neural Nets Outperform Boosted Trees on Tabular Data?," *NeurIPS,* pp. 76336-76369, 2023.

[8] H.-J. Ye, S.-Y. Liu, H.-R. Cai, Q.-L. Shou and D.-C. Zhan, "A Closer Look at Deep Learning Methods on Tabular Datasets," 2025.

[9] E. R. Kaempf, "Benchmarking 15 Machine Learning Models for Binary Classification: Accuracy, Complexity, and Speed," 2025.

[10] X. Huang, A. Khetan, M. Cvitkovic and Z. Karnin, "TabTransformer: Tabular Data Modeling Using Contextual Embeddings," *arXiv,* vol. 2012.06678v1, 2020.

[11] J. Komorniczak and P. Ksieniewicz, problexity - an open source Python library for binary classification problem complexity assessment, 2022.

[12] J. Eberlein, D. Rodriguez and R. Harrison, "The effect of data complexity on classifier performance," *Empirical Software Engineering,* vol. 30, no. 16, pp. 1-23, 2024.

[13] G. Armano and E. Tamponi, "Experimenting multiresolution analysis for identifying regions of different classification complexity," *Pattern Analysis and Applications,* vol. 19, no. 1, pp. 129-137, 2016.

[14] J. D. Pacual-Triana, D. Charte, M. A. Arroyo, A. Fernandez and F. Herrera, "Revisiting Data Complexity Metrics Based on Morphology for Overlap and Imbalance: Snapshot, New Overap Number of Balls Metrics and Singular Problems Prospect," *TBD,* vol. TBD, no. TBD, p. TBD, 2020.

[15] G. Zabergja, A. Kadra, C. M. Frey and J. Grabocka, "Tabular Data: Is Deep Learning All You Need," *TBD,* vol. TBD, p. TBD, TBD.

[16] H. O'Brian Quinn, M. Sedky, J. Francis and M. Streeton, "Literature Review of Explainable Data Analysis," *Electronics,* vol. 13, no. 19, p. 31, 2024.

[17] M. A. Ali, J. Liu, S. Moore and O. Nibouche, "Assessing the Effect of Data Complexity and Instance Overlap Issues on Imbalanced Learning," in *2024 the 7th International Conference on Big Data and Education*, Oxford, 2024.

[18] V. Borisov, T. Leemann, K. SeBler, J. Haug, M. Pawelczyk and G. Kasneci, "Deep Neural Networks and Tabular Data: A Survey," *arXiv preprint arXiv,* vol. 2110.01889, 2021.

[19] E. B. Mansilla and T. K. Ho, "On Classifier Domains of Competence," in *International Conference on Pattern Recognition*, 2004.